

Memoria Final

2ºDAW

Proyecto Intermodular
Desarrollo de Aplicaciones
Web

LINKEDLIST



**Jose Antonio
Ferrer Rodriguez**


**I.E.S. FERNANDO
AGUILAR QUIGNON**

ÍNDICE	1
1. INTRODUCCIÓN.	3
1.1. Propósito.	3
1.1.1. Descripción del problema que se trata de resolver	3
1.1.2. Necesidad a la que responde el proyecto	3
1.1.3. Ámbito del proyecto	3
1.2. Objetivos	4
1.2.1. Qué metas se pretenden alcanzar con el proyecto	4
1.3. Antecedentes	5
1.3.1. Qué hay actualmente similar al proyecto y su justificación	5
1.4. Viabilidad técnica y económica del proyecto	7
1.4.1. Esbozo de soluciones técnicas y viabilidad económica	7
1.4.2. Recursos hardware y software necesarios	8
1.4.3. Justificación de la solución elegida	8
1.5. Temporalización	8
1.5.1. Planificación temporal prevista sobre el desarrollo del proyecto	8
1.5.2. Actualizaciones respecto a la planificación inicial	9
2. ANÁLISIS	10
2.1. Documentación relevante	10
2.1.1. Resumen de documentación consultada	10
2.2. Definiciones	10
2.3. Requisitos funcionales y no funcionales	10
2.3.1. Requisitos funcionales	10
2.3.1.1. Requisitos funcionales de datos	10
2.3.1.2. Requisitos funcionales de contenido	11
2.3.1.3. Requisitos funcionales transaccionales	11
2.3.1.4. Requisitos funcionales de interfaz	11
2.3.2. Requisitos no funcionales	11
2.4. Casos de Uso. Diagramas UML de casos de uso	11
2.5. Modelado E/R. Diagramas E/R	12
2.6. Sketching de la interfaz (Prototipado)	13
2.6.1. Wireframing y Prototipado en Figma	13
3. Diseño	13
3.1. Arquitectura hardware y software de la solución	13
3.2. Modelado funcional de la solución. Diagramas de clase	14
3.3. Modelado de datos. Modelo relacional y Diccionario de Datos	14
3.3.1. Diccionario de Datos detallado	15
3.4. Prototipado de la interfaz	16
3.4.1. Prototipado usando Figma	16
4. IMPLEMENTACIÓN	19
4.1. Requisitos de instalación y ejecución	19
4.1.1. Entorno hardware y software necesario	19



4.2. Implementación funcional. Clases	20
4.3. Implementación del modelo de datos. Tablas	23
4.4. Implementación del Frontend (Angular)	26
4.4.1. Estructura de Componentes y Navegación	26
4.4.2. Servicios y Comunicación Asíncrona	27
4.4.3. Diseño Adaptativo y Experiencia de Usuario (UX)	28
4.4.4. Organización del Espacio de Trabajo	29
4.5. Seguridad y Control de Versiones	30
4.5.1. Implementación de Seguridad con JWT (Stateless)	30
4.5.2. Control de versiones con GIT	31
4.6. Despliegue y Virtualización (Docker)	31
5. PRUEBAS	33
5.1. Pruebas de módulos (Unitarias)	33
5.1.1 guardarUsuario_ok_cuandoNoExiste	33
5.1.2 guardarUsuario_lanzaExcepcion_siUsuarioExiste	34
5.1.3 guardarUsuario_debeCifrarPassword	35
5.1.4 login_exitoso_devuelveToken	35
5.1.5 login_fallido_siPasswordIncorrecta	36
5.1.6 actualizarUsuario_debeGuardarCambios	37
5.2. Pruebas de Integración	38
5.2.1 registrarUsuario_integracion_ok	38
5.2.2 contextLoads	38
5.2.3 listarUsuarios_integracion_debeRetornarLista	39
5.2.4 login_integracion_fallido_credencialesMalas	40
5.3. Pruebas de Carga y Rendimiento	40
5.3.1 Prueba de Concurrencia	41
5.3.2 Prueba de Rendimiento	42
5.3.3 Prueba de Estrés	42
6. SEGURIDAD	43
6.1. Mejoras de seguridad aplicadas	43
6.1.1. Cifrado de credenciales con BCrypt	43
6.1.2. Gestión de Autorización mediante JWT	44
6.1.3. Configuración de Políticas CORS y Permisos de Rutas	45
7. CONCLUSIONES	45
7.1. Grado de consecución de los objetivos inicialmente planteados	45
7.2. Dificultades encontradas	46
7.2.1. Técnicas y de otro tipo	46
8. PROPUESTAS DE MEJORA Y POSIBLES AMPLIACIONES	47
9. BIBLIOGRAFÍA Y RECURSOS ON-LINE	47
10. GLOSARIO DE TÉRMINOS	48
10.1. Informáticos	48
10.2. Problema	49
11. ANEXOS	49



1. INTRODUCCIÓN.

1.1. Propósito.

El propósito de este proyecto es ofrecer una solución tecnológica real a la gestión de información de productos en internet. A menudo, como usuarios, guardamos enlaces en blocs de notas, marcadores del navegador o mensajes de WhatsApp que luego se pierden o quedan desactualizados. Con este desarrollo, he buscado crear una plataforma donde esa información no solo se guarde, sino que se organice de forma inteligente mediante el uso de estructuras de datos optimizadas (LinkedList) y un sistema de recolección automática de datos.

1.1.1. Descripción del problema que se trata de resolver

Hoy en día, comprar online o seguirle la pista a ciertos productos es una tarea caótica. El problema no es la falta de información, sino el exceso de ella. Las tiendas online están diseñadas para que compres lo que ellas quieren, no para que tú organices tus necesidades.

Nos encontramos con tres problemas principales:

1. **La dispersión:** Los datos de un mismo tipo de producto están repartidos en decenas de webs con formatos distintos.
2. **La falta de personalización:** Las listas de "favoritos" de las grandes plataformas son muy rígidas y no permiten una clasificación propia o social.
3. **La dificultad de mantenimiento:** Introducir a mano cada característica de un producto es tedioso y lleva a que el usuario abandone la herramienta por falta de tiempo.

1.1.2. Necesidad a la que responde el proyecto

Este proyecto se justifica por la necesidad de una herramienta intermedia: algo que esté entre una simple hoja de Excel y una tienda online compleja. La idea es devolverle al usuario el control sobre su información.

Desde un punto de vista técnico, el proyecto responde a la necesidad de implementar soluciones de **rendimiento y escalabilidad**:

- **Gestión dinámica:** El uso de **LinkedList** no es casualidad; responde a la necesidad de gestionar listas donde los elementos cambian o se eliminan constantemente, permitiendo que la aplicación sea fluida incluso con mucho volumen de datos.
- **Automatización de carga:** Mediante técnicas de **scraping**, el sistema es capaz de "leer" webs y traer la información, ahorrando trabajo al usuario y dotando a la plataforma de un catálogo base funcional desde el primer minuto.
- **Despliegue moderno:** El uso de **Docker** responde a la necesidad de que el proyecto no se quede "atrapado" en mi ordenador, permitiendo que cualquier persona pueda ejecutarlo sin ser un experto en configurar bases de datos o servidores.



1.1.3. Ámbito del proyecto

Para que el proyecto fuera viable en el tiempo disponible, he acotado el alcance de la siguiente manera:

- **El alcance actual:** Un sistema completo donde el **Frontend** (Angular) y el **Backend** (Spring Boot) se comunican de forma fluida.
 - Un motor de persistencia robusto con **PostgreSQL**.
 - La lógica de gestión de listas basada en nodos ligados para una manipulación eficiente.
 - Un sistema de roles (Usuario y Administrador) para asegurar la integridad de los datos.
 - La contenedorización total para que la instalación sea "un solo clic".
- **Lo que podría ser en el futuro (Posibles extensiones):**
 - Se podría añadir un sistema de **webhooks** para avisar al usuario por correo si un producto de su lista cambia de precio.
 - Desarrollar una extensión de navegador que permita añadir productos a LinkedList directamente mientras navegas por Amazon o PCComponentes.
 - Añadir una capa de analítica para que el administrador vea qué tipos de productos son los más populares.
- **Lo que queda fuera:**
 - No se han incluido pasarelas de pago ni gestión de pedidos reales, ya que el enfoque es la gestión de la información, no la venta final.
 - No se contempla la integración con APIs de terceros que requieran suscripción de pago (se usa scraping propio por autonomía).

1.2. Objetivos

El desarrollo de LinkedList no solo busca ser una herramienta funcional, sino que persigue una serie de metas técnicas y de usuario que definen el éxito del proyecto.

1.2.1. Qué metas se pretenden alcanzar con el proyecto

Para dar una visión clara, los objetivos se han dividido en tres categorías:

A. Objetivos Estratégicos (Usuario)



- **Centralización de la información:** Lograr que el usuario reduzca el tiempo de búsqueda de productos en un 60%, al tener sus referencias unificadas en un solo panel.
- **Fomento del consumo inteligente:** Proporcionar herramientas de valoración y comentarios que permitan al usuario tomar decisiones basadas en datos y experiencias de otros miembros, evitando compras impulsivas o de baja calidad.
- **Optimización de la organización personal:** Permitir la creación de listas temáticas ilimitadas, gestionadas de forma fluida y sencilla.

B. Objetivos Técnicos (Desarrollo)

- **Eficiencia en estructuras de datos:** Implementar y demostrar la viabilidad de las **LinkedList** (listas enlazadas) para la gestión dinámica de elementos en memoria, asegurando que las operaciones de reordenación e inserción sean óptimas.
- **Automatización mediante Scraping:** Desarrollar un motor de recolección de datos capaz de extraer información precisa (precios, imágenes, descripciones) de fuentes externas de forma autónoma.
- **Arquitectura Robusta y Escalable:** Diseñar un sistema basado en microservicios (contenedorizados con Docker) que permita el crecimiento del catálogo sin degradar el rendimiento del servidor.

C. Objetivos de Seguridad y Calidad

- **Protección de la privacidad:** Implementar un sistema de autenticación basado en **JSON Web Tokens (JWT)** y asegurar que el tratamiento de datos cumple con la normativa vigente ([RGPD](#) y [LOPD-GDD](#)).
- **Alta Disponibilidad:** Garantizar que la aplicación sea accesible desde cualquier dispositivo mediante un diseño **Responsive** y un despliegue en la nube o local mediante contenedores.

1.3. Antecedentes

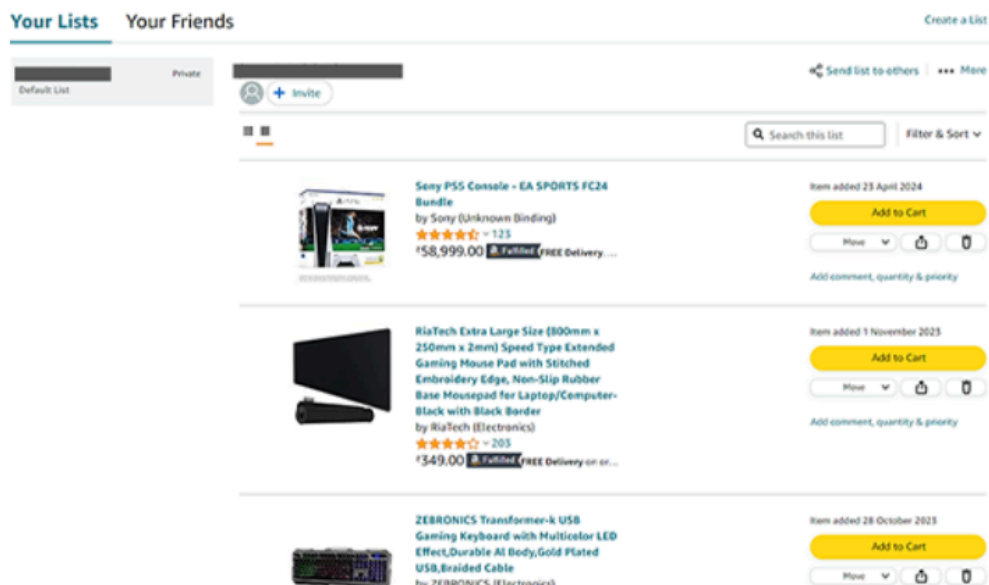
Para el desarrollo de **LinkedList**, se ha realizado un estudio previo de las herramientas de gestión de compras y listas de deseos actuales. El objetivo es identificar las carencias de las grandes plataformas y justificar la creación de una alternativa independiente y personalizada.

1.3.1. Qué hay actualmente similar al proyecto y su justificación

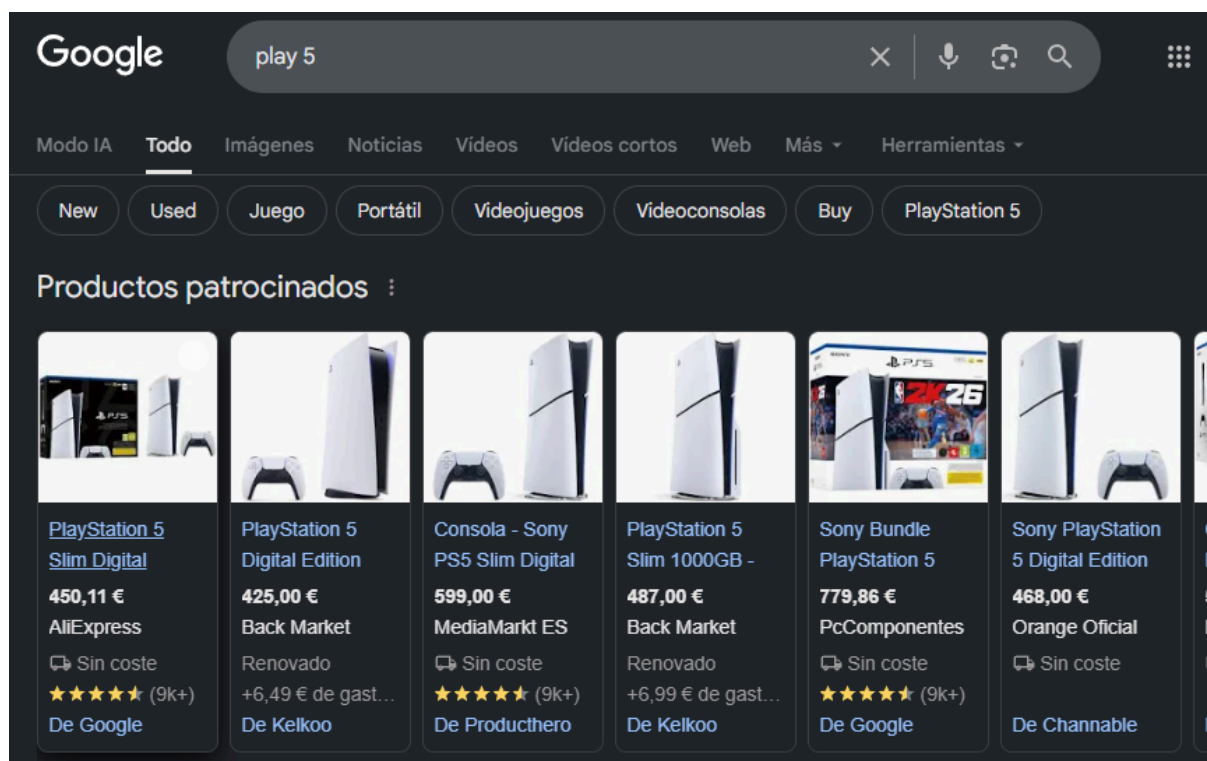
En el mercado actual predominan tres tipos de soluciones, pero ninguna cubre la necesidad integral que plantea este proyecto:

- **Listas nativas de E-commerce (Ej. Amazon Wishlist):** Son las más utilizadas, pero presentan un conflicto de intereses evidente: solo permiten añadir productos de su propio catálogo. El usuario se ve "atrapado" en un ecosistema cerrado, lo que impide una comparación objetiva con competidores externos.



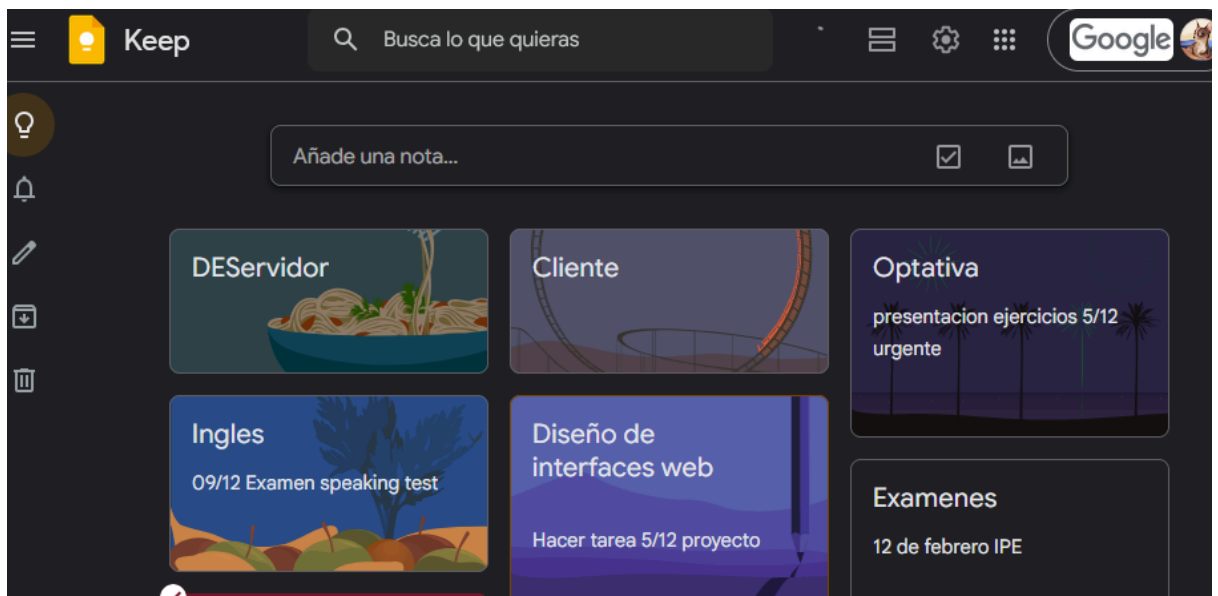


- **Agregadores de ofertas (Ej. Google Shopping):** Aunque son excelentes para encontrar precios bajos, funcionan de forma algorítmica y generalista. No permiten al usuario crear un espacio personal y privado donde organizar sus futuros gastos o proyectos personales de forma estructurada.



- **Aplicaciones de notas genéricas (Ej. Google Keep):** Muchos usuarios recurren a estas para guardar enlaces. Sin embargo, carecen de automatización. El usuario debe escribir a mano el nombre, precio e imagen, lo que convierte el mantenimiento de la lista en una tarea tediosa que suele acabar en el abandono de la herramienta.





Justificación del desarrollo de LinkedList: Mi proyecto justifica su existencia al cerrar la brecha entre estas opciones. A diferencia de las anteriores, LinkedList ofrece:

1. **Independencia Total:** El usuario puede centralizar productos de cualquier origen.
2. **Automatización:** Gracias al motor de *scraping*, la información se precarga sola, eliminando la fricción de la entrada de datos manual.
3. **Privacidad y Orden:** Un entorno sin publicidad agresiva, centrado exclusivamente en la gestión eficiente de los datos mediante estructuras de nodos.

1.4. Viabilidad técnica y económica del proyecto

En este apartado se analiza la factibilidad de llevar a cabo el proyecto LinkedList, evaluando tanto las herramientas tecnológicas como los costes asociados.

1.4.1. Esbozo de soluciones técnicas y viabilidad económica

Se ha optado por una arquitectura de **microservicios contenedorizados**, lo que permite separar el Frontend (Angular) del Backend (Spring Boot). Esta solución garantiza que, si una parte del sistema necesita actualizarse, no afecte al resto.

Viabilidad Económica: El proyecto es altamente viable ya que se basa íntegramente en el ecosistema **Open Source** (Código Abierto).

- **Coste de licencias:** 0€, al utilizar tecnologías gratuitas como Java, PostgreSQL y Angular.
- **Coste de infraestructura:** Mínimo, ya que para la fase de explotación basta con un servidor VPS básico o servicios en la nube.
- **Coste de desarrollo:** Se estima únicamente en horas



1.4.2. Recursos hardware y software necesarios

Para asegurar el éxito del proyecto en las fases de desarrollo y explotación, se requieren los siguientes elementos:

Hardware:

- Procesador con soporte para virtualización (mínimo 4 núcleos).
- Memoria RAM mínima de 8GB (recomendado 16GB para fluidez con Docker).
- Almacenamiento SSD para optimizar el acceso a la base de datos.

Software y Tecnologías (Explotación y Desarrollo):

- **Frontend:** Angular 17+ con Bootstrap para el diseño *Responsive*.
- **Backend:** Java 17 con Spring Boot 3.
- **Base de Datos:** PostgreSQL (Relacional).
- **Contenedores:** Docker y Docker Compose para la orquestación.
- **Seguridad:** Protocolo HTTPS y JSON Web Tokens (JWT).
- **Recolección de datos:** Web Scraping propio.
- **Github:** control de versiones

1.4.3. Justificación de la solución elegida

La elección de este *stack* tecnológico no es aleatoria, responde a las siguientes necesidades del proyecto:

1. **Docker:** Se elige para garantizar la portabilidad absoluta. Como se ha demostrado en las pruebas de despliegue, permite que la aplicación funcione en cualquier sistema sin conflictos de versiones.
2. **Spring Boot:** Proporciona una robustez necesaria para manejar el *DataSeeder* y la lógica compleja de las listas enlazadas.
3. **PostgreSQL:** Se prefiere sobre otras bases de datos por su excelente manejo de datos relacionales complejos y su estabilidad en entornos Docker.
4. **Github:** Para el control de versiones fácil y seguro, con la facilidad de uso que tiene y lo grande que puede llegar a ser el proyecto

1.5. Temporalización

El desarrollo del proyecto se ha estructurado en el periodo del curso académico, partiendo desde octubre hasta mayo, siguiendo una metodología ágil simplificada para permitir ajustes rápidos tras cada fase de entrega.

1.5.1. Planificación temporal prevista sobre el desarrollo del proyecto

La hoja de ruta del proyecto se dividió en las siguientes fases:



- **Análisis y Diseño (3 semanas):** Definición de requisitos, creación del modelo Entidad-Relación y prototipado de interfaces en Figma.
- **Configuración del Backend y Base de Datos (4 semanas):** Creación de la estructura del proyecto en Spring Boot, configuración de PostgreSQL y desarrollo del modelo de datos inicial.
- **Desarrollo de Lógica de Negocio y Scraping (4 semanas):** Implementación de la gestión de listas mediante la estructura LinkedList, seguridad con JWT y el motor de recolección de datos (Scraper).
- **Desarrollo del Frontend (3 semanas):** Creación de componentes en Angular, integración de Bootstrap para diseño responsive y conexión con el servicio API.
- **Contenedorización, Pruebas y Despliegue (3 semanas):** Creación de imágenes Docker, orquestación con Docker Compose y testeo en diferentes entornos de ejecución.

1.5.2. Actualizaciones respecto a la planificación inicial

Como en todo proceso de desarrollo real, surgieron imprevistos que obligaron a modificar la planificación original para asegurar la calidad de la entrega final:

- **Ajuste en la Gestión de Recursos Estáticos:** Inicialmente se previó que el scraping descargara imágenes en tiempo real. Sin embargo, para mejorar la velocidad y evitar bloqueos de IPs externas, se decidió implementar un *DataSeeder* que precarga archivos locales (como los .svg). Esto retrasó una semana la fase de Backend pero mejoró mucho la estabilidad.
- **Complejidad en la Portabilidad con Docker:** El despliegue en diferentes sistemas operativos reveló problemas de permisos de usuario y rutas de archivos. Esto obligó a dedicar una semana extra a la depuración de los archivos *Dockerfile* y a la creación de una guía de solución de problemas para el usuario final (como la aceptación de certificados SSL).
- **Optimización de la Interfaz:** Tras las primeras pruebas de usuario, se decidió añadir una sección de FAQ (Preguntas Frecuentes) y un formulario de contacto. Estos elementos no estaban previstos inicialmente, pero se añadieron para cumplir con los estándares de usabilidad y mejorar la experiencia de usuario.

2. ANÁLISIS

En esta fase se detalla el funcionamiento interno de LinkedList, definiendo qué necesidades cubre (requisitos) y cómo se estructura la información y la interacción del usuario (modelado y diseño).

2.1. Documentación relevante

Para el diseño de **LinkedList**, se ha recopilado información basada en las necesidades reales de usuarios que realizan compras recurrentes en internet.

2.1.1. Resumen de documentación consultada

Se han analizado las estructuras de las "Listas de deseos" de plataformas como Amazon y PCComponentes, extrayendo los campos de datos necesarios para que un producto sea identificable (precio, imagen, descripción). También se han consultado manuales de buenas prácticas de la W3C para asegurar que la interfaz sea accesible y el [estándar ISO/IEC 27001](#) para la gestión de la seguridad de la información.

2.2. Definiciones

Siguiendo las recomendaciones para facilitar la lectura técnica del proyecto, se aclaran los siguientes conceptos:

- **Estructura LinkedList:** Tipo de dato lineal donde cada elemento apunta al siguiente, permitiendo una gestión de memoria dinámica y eficiente para listas que cambian constantemente.
- **Web Scraping:** Proceso automatizado mediante software que extrae datos de sitios web para cargarlos en nuestra propia base de datos.
- **JWT (JSON Web Token):** Método para transmitir de forma segura información entre el cliente y el servidor como un objeto JSON, garantizando que el usuario está autenticado.
- **DataSeeder:** Script o función encargada de rellenar la base de datos con información inicial para que la aplicación sea funcional desde el primer despliegue.

2.3. Requisitos funcionales y no funcionales

2.3.1. Requisitos funcionales

2.3.1.1. Requisitos funcionales de datos

- El sistema guardará el nombre, correo electrónico y contraseña (cifrada) de cada usuario registrado.
- Se almacenará la URL original del producto, su precio, descripción y una imagen asociada.
- El sistema registrará la fecha de creación de cada lista y la categoría a la que pertenece.



2.3.1.2. Requisitos funcionales de contenido

- Cada producto tendrá asociada una imagen para facilitar su identificación visual.
- Los usuarios tendrán un avatar o imagen de perfil personalizable para fomentar la identificación con la plataforma.

2.3.1.3. Requisitos funcionales transaccionales

- Los usuarios pueden registrarse e iniciar sesión mediante credenciales seguras.
- El usuario podrá crear, editar, reordenar y eliminar listas de productos en tiempo real.
- El sistema permitirá añadir productos mediante un formulario o mediante el motor de scraping automático.
- El administrador podrá gestionar (borrar o editar) cualquier producto o usuario que incumpla las normas de la plataforma.

2.3.1.4. Requisitos funcionales de interfaz

- La plataforma brindará un formulario de contacto para soporte técnico.
- El sistema ofrecerá una sección de FAQ (Preguntas Frecuentes) para resolver dudas de uso rápidamente.
- Se dispondrá de un mecanismo claro para recuperar o modificar la contraseña del usuario.

2.3.2. Requisitos no funcionales

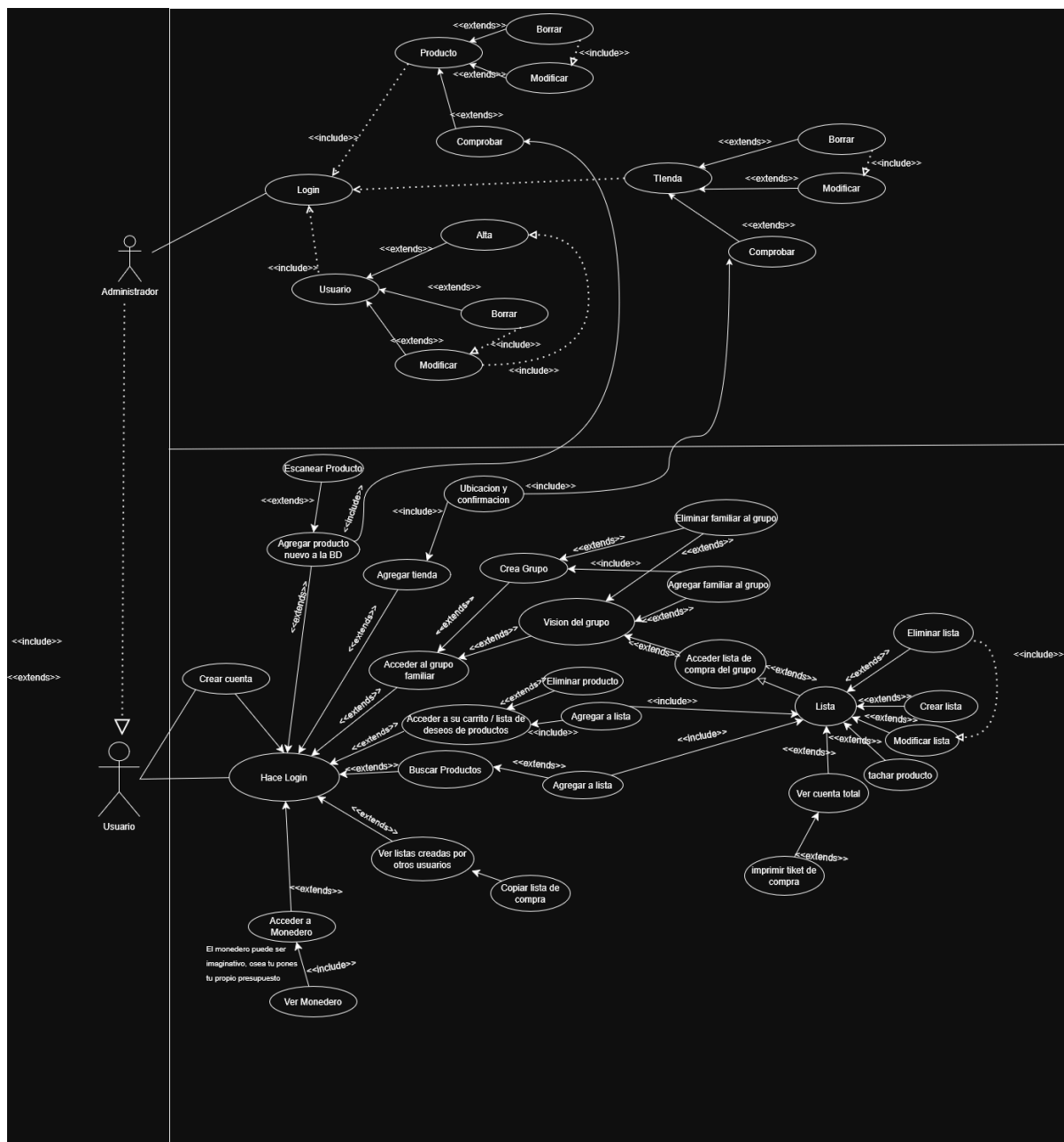
- **Seguridad y LOPD:** Debido a que se guardan datos de usuario, el sistema cumple con la [Ley Orgánica 3/2018 \(LOPD-GDD\)](#) y el [RGPD](#), asegurando que los datos están protegidos y cifrados.
- **Usabilidad:** La interfaz debe ser Responsive, adaptándose correctamente a dispositivos móviles, tablets y ordenadores.
- **Rendimiento:** El tiempo de respuesta del motor de búsqueda y carga de listas no debe superar los 4 segundos para garantizar una buena experiencia de usuario.
- **Disponibilidad:** El uso de HTTPS garantiza que la comunicación entre el cliente y el servidor sea privada y segura ante ataques de interceptación.

2.4. Casos de Uso. Diagramas UML de casos de uso

Para definir el comportamiento del sistema, se han identificado dos actores principales: el Usuario Registrado, que gestiona sus propias listas, y el Administrador, que vela por el mantenimiento y la integridad de la plataforma.

Los casos de uso principales incluyen la gestión de perfiles, la creación dinámica de listas y el proceso de scraping para la obtención automática de datos de productos.





- **Actor Usuario:** Puede registrarse, iniciar sesión, crear listas, añadir productos (manual/scraping) y modificar su perfil.
- **Actor Administrador:** Posee los permisos del usuario y añade la capacidad de gestionar el catálogo global y moderar usuarios.

2.5. Modelado E/R. Diagramas E/R

El diseño de la base de datos se ha realizado buscando la normalización para evitar redundancias. El núcleo del modelo es la relación entre los **Usuarios** y sus **Listas**, las cuales contienen **Productos**.



La estructura permite que muchos usuarios tengan múltiples listas (N:M), compartiéndola y que cada lista contenga múltiples productos, manteniendo la integridad referencial mediante claves foráneas.

- **Entidad Usuario:** Almacena credenciales y datos de perfil.
- **Entidad Lista:** Vincula al usuario con un conjunto de productos y guarda metadatos como la fecha de creación.
- **Entidad Producto:** Contiene la información técnica (precio, imagen, URL) obtenida mediante el proceso de scraping o inserción manual.

2.6. Sketching de la interfaz (Prototipado)

El diseño de la interfaz de **LinkedList** se ha centrado en la experiencia de usuario (UX) y en la limpieza visual. Para ello, se ha utilizado la herramienta de prototipado **Figma**, que ha permitido definir la disposición de los elementos y la navegación antes de pasar a la fase de implementación con Angular.

2.6.1. Wireframing y Prototipado en Figma

Dada la naturaleza del proyecto, se optó por un diseño funcional basado en componentes. El proceso se centró en dos vistas principales: el panel de gestión de listas y la vista de detalle de productos obtenidos mediante scraping.

3. Diseño

En este capítulo se describe la arquitectura técnica, la lógica de objetos y la estructura de persistencia que dan soporte a **LinkedList**. El diseño se ha orientado a la modularidad y la alta disponibilidad mediante el uso de microservicios.

3.1. Arquitectura hardware y software de la solución

La solución implementa una arquitectura de **tres capas** (Presentación, Negocio y Datos) totalmente desacopladas mediante el uso de contenedores Docker.

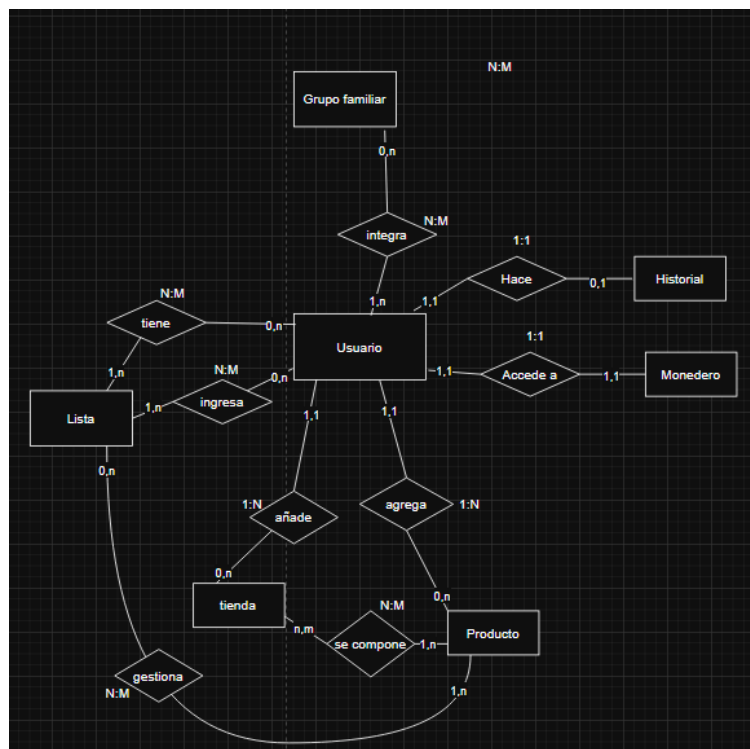
- **Arquitectura Software:**
 - **Frontend (Capa de Presentación):** Desarrollada con Angular 17, utilizando una arquitectura basada en componentes y servicios. Se implementa Bootstrap 5 para el motor de estilos.



- **Backend (Capa de Negocio):** Basado en Spring Boot 3. Se utiliza Spring Security para la gestión de filtros de seguridad y Hibernate como motor de mapeo objeto-relacional .
- **Orquestación y Despliegue:** Se emplea Docker Compose para definir redes virtuales privadas entre los contenedores, asegurando que la base de datos solo sea accesible desde el backend.
- **Arquitectura Hardware:**
 - El sistema es compatible con procesadores de arquitectura x64 y ARM64 (gracias a las imágenes multi-plataforma de Docker). Se recomienda un entorno con al menos 4GB de memoria RAM disponible para garantizar la fluidez de los tres servicios simultáneos.

3.2. Modelado funcional de la solución. Diagramas de clase

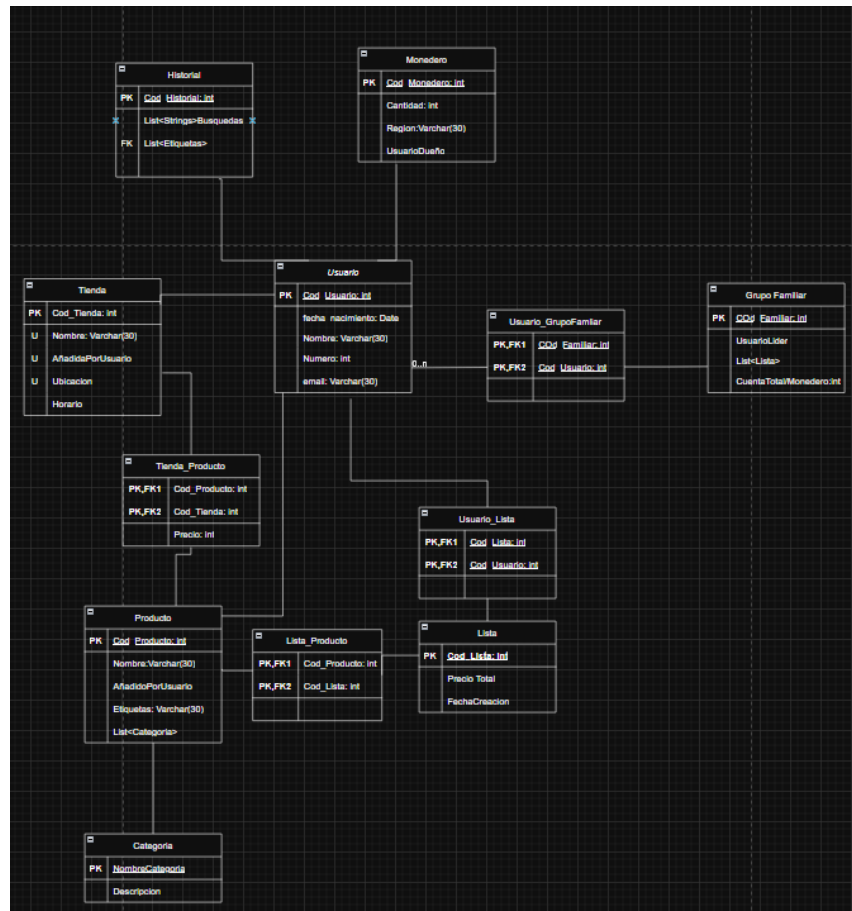
Para la construcción del sistema se ha seguido el paradigma de Programación Orientada a Objetos (POO). El diagrama de clases representa la jerarquía y las relaciones entre los controladores, servicios y entidades.



3.3. Modelado de datos. Modelo relacional y Diccionario de Datos



El diseño de la base de datos se ha normalizado hasta la Tercera Forma Normal (3FN) para eliminar redundancias y asegurar la integridad de la información de los usuarios.



3.3.1. Diccionario de Datos detallado

A continuación, se describen técnicamente **(solo los datos indispensables de cada tabla)** las tablas principales del sistema :

- Tabla: users
 - id (BIGINT, PK): Identificador único autoincremental.
 - username (VARCHAR 50, UNIQUE): Nombre de usuario para el acceso.
 - password (VARCHAR 255): Hash de la contraseña (encriptada).
 - email (VARCHAR 100, UNIQUE): Correo electrónico de contacto y recuperación.
 -

- Tabla: lists
 - id (BIGINT, PK): Identificador de la lista.



- name (VARCHAR 100): Nombre descriptivo de la lista de productos.
- user_id (BIGINT, FK): Referencia al propietario de la lista (Relación 1:N).
- created_at (TIMESTAMP): Fecha y hora de creación automática.
-
- Tabla: products
 - id (BIGINT, PK): Identificador único del producto.
 - title (VARCHAR 255): Nombre del producto extraído mediante scraping.
 - price (NUMERIC 10,2): Precio con precisión decimal para moneda.
 - image_url (TEXT): Ruta o URL de la imagen del producto.
 - product_url (TEXT): Enlace directo a la tienda de origen.
 - list_id (BIGINT, FK): Lista a la que pertenece el producto.
 -

3.4. Prototipado de la interfaz

El diseño visual busca la eficiencia operativa, reduciendo el número de clics necesarios para que el usuario organice su información.

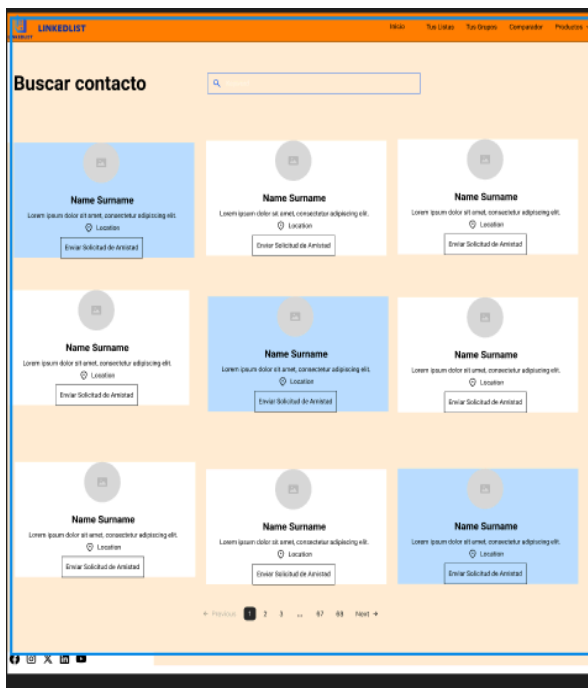
3.4.1. Prototipado usando Figma

El prototipo final define no solo la estética, sino también el comportamiento dinámico de la aplicación (transiciones, estados de carga y validaciones de formularios).





PRODUCTOS

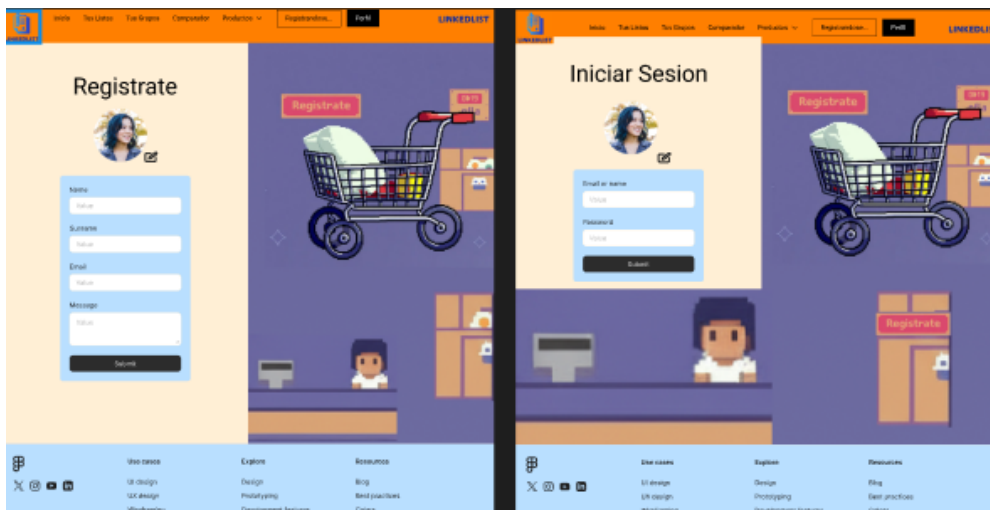


Ahora paso a ser la seccion COMENTARIOS





Pagina de INICIO



LOGIN/REGISTRO





MIS LISTAS

4. IMPLEMENTACIÓN

En este apartado se detalla la construcción física del sistema, los requisitos para su despliegue y la lógica de programación empleada en los componentes clave de la aplicación.

4.1. Requisitos de instalación y ejecución

Para garantizar que **LinkedList** funcione correctamente, se ha optado por una solución basada en Docker, lo que elimina la necesidad de instalar dependencias como Java o PostgreSQL de forma manual en el equipo anfitrión.

4.1.1. Entorno hardware y software necesario

Se definen los requisitos mínimos y recomendados para asegurar la fluidez de la aplicación (Frontend, Backend y Base de Datos corriendo simultáneamente):

Requisitos de Software:

- **Sistema Operativo:** Windows 10/11 , macOS o distribuciones Linux (Ubuntu 22.04+ recomendado).
- **Motor de Contenedores:** Docker Desktop o Docker Engine con el plugin Docker Compose instalado.
- **Navegador Web:** Versiones actualizadas de Google Chrome, Mozilla Firefox o Microsoft Edge para la correcta visualización del CSS y ejecución de Angular.

Requisitos de Hardware:

- **Procesador:** Mínimo 2 núcleos (4 recomendados) con soporte para virtualización activado en BIOS.



- **Memoria RAM:** Mínimo 4GB libres (8GB recomendables para el entorno de ejecución de contenedores).
- **Almacenamiento:** 2GB de espacio libre en disco para las imágenes de Docker y los datos

4.2. Implementación funcional. Clases

La lógica de negocio se ha estructurado siguiendo los principios de Clean Code y el patrón por capas en el Backend (Spring Boot). A continuación, se describen los controladores y servicios que gestionan las funcionalidades principales:

- **Autenticación (controllerUsuario):** Gestiona el registro y login mediante JWT. Se encarga de la validación de credenciales y la emisión de tokens seguros.

```
// ----- LOGIN -----
// Loguea al usuario y devuelve sus datos y el token. (Tiene que recibir nombre y contraseña)
@PostMapping("/login")
public ResponseEntity<> login(@RequestBody Map<String, String> request) {
    String ident = request.get("nombre"); // nombre o email
    String password = request.get("password");

    if (ident == null || ident.trim().isEmpty()) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body(Map.of("error", "El nombre de usuario o email es requerido"));
    }

    if (password == null || password.trim().isEmpty()) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body(Map.of("error", "La contraseña es requerida"));
    }

    try {
        AuthResponse authResponse = serviceAuthen.login(ident, password);
        return ResponseEntity.ok(authResponse);
    } catch (RuntimeException e) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
            .body(Map.of("error", e.getMessage()));
    }
}

// ----- RENOVAR TOKEN -----
// Te da un token nuevo si el que tienes es válido. (Tiene que recibir el token actual)
@GetMapping("/renovar")
public ResponseEntity<AuthResponse> obtenerNuevoJWT(@RequestHeader("Authorization") String authHeader) {
    String token = serviceJWT.limpiarToken(authHeader);
    String nombre = serviceJWT.obtenerSubject(token);
    String rol = serviceJWT.obtenerRol(token);
    Long id = serviceJWT.obtenerId(token);
    AuthResponse nuevoToken = serviceAuthen.renovarToken(nombre, rol, id);
    return ResponseEntity.ok(nuevoToken);
}
```

- **Gestión de Productos (controllerProducto):** Implementa los endpoints para la búsqueda, filtrado, creación y rechazo de productos. Aquí reside la lógica que conecta con el scraping.



```
// ----- Confirmar producto -----
// Confirma un producto. (Tiene que recibir el Token y el ID)
@PostMapping("/confirm/{id}")
public ResponseEntity<> confirmarProducto(
    @PathVariable Long id,
    @RequestHeader("Authorization") String authHeader) {

    String token = extraerToken(authHeader);
    if (token == null || !jwtService.esTokenValido(token)) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Token inválido o ausente");
    }

    Usuario admin = encontrarUsuarioPorNombre(jwtService.obtenerSubject(token));
    if (admin == null)
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Usuario no encontrado");

    try {
        ProductoDTO confirmado = service.confirmarProducto(id, admin);
        return ResponseEntity.ok(confirmado);
    } catch (RuntimeException e) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN).body(e.getMessage());
    }
}

// ----- Rechazar producto -----
// Rechaza un producto. (Tiene que recibir el Token y el ID)
@DeleteMapping("/reject/{id}")
public ResponseEntity<> rechazarProducto(
    @PathVariable Long id,
    @RequestHeader("Authorization") String authHeader) {

    String token = extraerToken(authHeader);
    if (token == null || !jwtService.esTokenValido(token)) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Token inválido o ausente");
    }

    Usuario admin = encontrarUsuarioPorNombre(jwtService.obtenerSubject(token));
    if (admin == null)
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Usuario no encontrado");

    try {
        boolean ok = service.rechazarProducto(id, admin);
        return ok ? ResponseEntity.noContent().build()
            : ResponseEntity.status(HttpStatus.NOT_FOUND).body("Producto no encontrado");
    } catch (RuntimeException e) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN).body(e.getMessage());
    }
}
}
```

- **Gestión de Listas (controllerLista):** Es el núcleo de la aplicación, permitiendo la creación de estructuras dinámicas de productos vinculadas a cada usuario.

```
// ----- LISTAR PÚBLICAS -----
@GetMapping("/publicas")
public ResponseEntity<List<ListaDetalladoDTO>> listarPublicas() {
    return ResponseEntity.ok(service.obtenerListasPublicas());
}

// ----- PUBLICAR/DESPUBLICAR -----
@PostMapping("/{id}/publicar")
public ResponseEntity<> publicarLista(@PathVariable Long id, @RequestHeader("Authorization") String authHeader) {
    String token = jwtService.limpiarToken(authHeader);
    if (token == null || !jwtService.esTokenValido(token))
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Token inválido o ausente");
    String rol = jwtService.obtenerRol(token);
    ListaDTO lista = service.obtenerListaDTO(id);
    if (lista == null)
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body("Lista no encontrada");

    if (!"ADMIN".equalsIgnoreCase(rol)) {
        Long userId = jwtService.obtenerId(token);
        if (userId == null || !lista.getUsuarioDueñoId().equals(userId)) {
            return ResponseEntity.status(HttpStatus.FORBIDDEN)
                .body("Solo el dueño o admin pueden publicar esta lista");
        }
    }

    boolean ok = service.cambiarEstadoPublicacion(id, estado: true);
    return ok ? ResponseEntity.ok().build() : ResponseEntity.notFound().build();
}
}
```



- **Gestión de Perfil (controllerPerfil):** Permite al usuario actualizar sus datos personales, avatar y preferencias de seguridad.

```
@PostMapping
public ResponseEntity<?> crear(
    @Valid @RequestBody CrearPerfilRequestDTO perfilDTO,
    @RequestHeader("Authorization") String authHeader) {

    // limpiar y validar token
    String token = serviceJWT.limpiarToken(authHeader);
    if (token == null || !serviceJWT.esTokenValido(token)) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
            .body("Token inválido o ausente");
    }

    // Obtener datos del token
    Long idToken = serviceJWT.obtenerId(token);
    String rol = serviceJWT.obtenerRol(token);

    // Verificar permisos: ADMIN o el propio usuario
    if (!"ADMIN".equalsIgnoreCase(rol) && !idToken.equals(perfilDTO.getUsuarioId())) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN)
            .body("No puedes crear un perfil para otro usuario");
    }

    // Crear perfil usando la instancia correcta del service
    try {
        PerfilUsuarioDTO creado = service.guardarPerfil(perfilDTO);
        creado.setEmail(perfilDTO.getEmail());
        return ResponseEntity.status(HttpStatus.CREATED).body(creado);
    } catch (Exception e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body("Error al crear perfil: " + e.getMessage());
    }
}
```

- **Gestion de Comentarios (controllerComentario):** Permite al usuario crear y hacer valoraciones de productos para tomar referencia otros usuarios.



Todos esos controller, tienen su propio servicio, modelo, repo y dtos (de entrada y de salida):

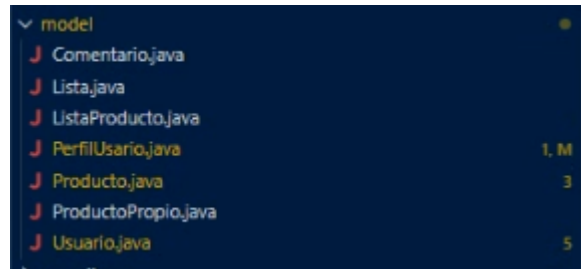
```
C:\.
├── controller
│   ├── controllerComentario.java
│   ├── controllerContacto.java
│   ├── controllerLista.java
│   ├── controllerPerfilUsuario.java
│   ├── controllerProducto.java
│   ├── controllerProductoPropio.java
│   ├── controllerUsuario.java
│   └── TestController.java
├── dto
│   ├── AuthResponse.java
│   ├── ComentarioDTO.java
│   ├── CrearComentarioDTO.java
│   ├── CrearListaRequestDTO.java
│   ├── CrearPerfilRequestDTO.java
│   ├── CrearProductoDTO.java
│   ├── CrearProductoPropioDTO.java
│   ├── CrearUsuarioRequestDTO.java
│   ├── ListaDetalleDTO.java
│   ├── ListaDTO.java
│   ├── PerfilUsuarioDTO.java
│   ├── ProductoDTO.java
│   ├── ProductoEstadoDTO.java
│   ├── ProductoPropioDTO.java
│   ├── UnirseListaRequestDTO.java
│   ├── UsuarioDTO.java
│   └── UsuarioMinimoDTO.java
├── exception
│   ├── ConflictException.java
│   ├── DuplicateResourceException.java
│   ├── GlobalExceptionHandler.java
│   └── RecursoDuplicadoException.java
├── model
│   ├── Comentario.java
│   ├── Lista.java
│   ├── ListaProducto.java
│   ├── PerfilUsuario.java
│   ├── Producto.java
│   ├── ProductoPropio.java
│   └── Usuario.java
├── repository
│   ├── repositoryComentario.java
│   ├── repositoryLista.java
│   ├── repositoryListaProducto.java
│   ├── repositoryPerfilUsuario.java
│   ├── repositoryProducto.java
│   ├── repositoryProductoPropio.java
│   └── repositoryUsuario.java
└── service
    ├── DataSeeder.java
    ├── EmailService.java
    ├── JwtAuthenticationFilter.java
    ├── SecurityConfig.java
    ├── serviceAuthen.java
    ├── serviceComentario.java
    ├── serviceMT.java
    ├── serviceLista.java
    ├── servicePerfilUsuario.java
    ├── serviceProducto.java
    ├── serviceProductoPropio.java
    └── serviceUsuario.java
```

4.3. Implementación del modelo de datos. Tablas

La persistencia de la aplicación se gestiona de forma automática mediante **JPA** (Java Persistence API) y **Hibernate**. En lugar de escribir scripts SQL manuales, se han definido clases Entidad en Java que dictan la estructura física de la base de datos PostgreSQL.

Este enfoque asegura que el modelo de datos y el código estén siempre sincronizados, aplicando restricciones de integridad (como nullable = false o unique = true) directamente desde el Backend.





```
@Entity
@Table(name = "usuario")
@Data
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "El nombre de usuario no puede ser nulo")
    @Size(min = 2, max = 255, message = "El nombre debe tener entre 2 y 255 caracteres")
    @Column(unique = true)
    private String nombre;

    @NotBlank(message = "El email no puede ser nulo")
    @Size(min = 5, max = 255, message = "El email debe tener entre 5 y 255 caracteres")
    private String email;

    @NotBlank(message = "La contraseña no puede ser nula")
    @Size(min = 4, max = 255, message = "La contraseña debe tener entre 4 y 255 caracteres")
    private String contraseña;

    @NotBlank(message = "El rol no puede ser nulo")
    private String rol;

    @PastOrPresent(message = "La fecha de registro no puede ser futura")
    private LocalDate fechaRegistro;

    private Boolean activo = true;

    // Campos para recuperación de contraseña
    private String resetToken;
    private LocalDateTime resetTokenExpiration;

    // Campos para verificación de email
    private String verificationToken;

    @OneToMany(mappedBy = "usuario", cascade = CascadeType.ALL, orphanRemoval = true)
    @JsonManagedReference(value = "usuario-comentarios")
    private List<Comentario> comentarios = new ArrayList<>();

    @OneToMany(mappedBy = "usuarioRegistrador", cascade = CascadeType.ALL, orphanRemoval = true)
    @JsonManagedReference(value = "usuario-productos")
    private List<Producto> listaProductosSubidos;
```

Esto solo es un fragmento de todo el modelo usuario

Estrategia de Población de Datos (DataSeeder): Para que la aplicación no se despliegue vacía, se ha implementado un servicio de Seeding. He utilizado la interfaz CommandLineRunner de Spring Boot, que ejecuta un bloque de código automáticamente al arrancar el servidor. Este proceso lee los recursos locales y utiliza los Repositorios de Spring Data para realizar las inserciones iniciales.



```

39 public class DataSeeder implements CommandLineRunner {
298     }
299
300     private void cargarDesdeCsv(String csvPath, String supermercado, Set<String> productosExistentes) {
301         Path archivoCSV = buscarArchivo(csvPath);
302         if (archivoCSV == null || !Files.exists(archivoCSV)) {
303             logger.error("No se encontró el archivo CSV de " + supermercado + ": " + csvPath);
304             return;
305         }
306
307         logger.info("Importando desde: " + archivoCSV.toAbsolutePath() + " (" + supermercado + ")");
308         Map<String, Producto> mapaProductos = new java.util.HashMap<>();
309         int saltados = 0;
310
311         try (BufferedReader br = new BufferedReader(
312             new InputStreamReader(new FileInputStream(archivoCSV.toFile()), StandardCharsets.UTF_8))) {
313             String line;
314             boolean firstLine = true;
315             while ((line = br.readLine()) != null) {
316                 if (firstLine) {
317                     firstLine = false;
318                     continue;
319                 }
320                 if (line.trim().isEmpty())
321                     continue;
322
323                 try {
324                     String[] values = line.split(",(?=[^\\"]*(\\\"[^\\"]*\")*[^\\"]*$)");
325                     if (values.length >= 5) {
326                         String nombre = values[0].trim().replace("\\\"", "");
327                         String precioCrudo = values[1].trim().replace("\\\"", "");
328                         String imageUrl = values[2].trim().replace("\\\"", "");
329                         String url = values[3].trim().replace("\\\"", "");
330                         String categoriaCruda = values[4].trim().replace("\\\"", "");
331
332                         String nombreNormalizado = nombre.length() > 100 ? nombre.substring(0, 100) : nombre;
333                         String clave = (nombreNormalizado + "|" + supermercado).toLowerCase();
334
335                         if (productosExistentes.contains(clave)) {
336                             saltados++;
337                             continue;
338                         }
339
340                         String nombreNormalizado = nombre.length() > 100 ? nombre.substring(0, 100) : nombre;
341                         String clave = (nombreNormalizado + "|" + supermercado).toLowerCase();
342
343                         if (productosExistentes.contains(clave)) {
344                             saltados++;
345                             continue;
346                         }
347
348                         Producto p;
349                         if (mapaProductos.containsKey(clave)) {
350                             p = mapaProductos.get(clave);
351                         } else {
352                             p = new Producto();
353                             p.setNombre(nombreNormalizado);
354                             p.setSupermercado(supermercado);
355                             p.setConfirmado(confirmado: true);
356                             p.setImageUrl(imageUrl.length() > 255 ? imageUrl.substring(0, 255) : imageUrl);
357                             String urlFinal = url.isEmpty() ? "No disponible" : url;
358                             String desc = "Producto importado de " + supermercado + ". Ver en: " + urlFinal;
359                             p.setDescripcion(desc.length() > 300 ? desc.substring(0, 300) : desc);
360
361                             double precioFinal = 1.99;
362                             if (!precioCrudo.isEmpty()) {
363                                 String[] partesPrecio = precioCrudo.split(" ");
364                                 if (partesPrecio.length > 0) {
365                                     String numStr = partesPrecio[0].replace(",", ".").replaceAll("[^0-9.]", "");
366                                     try {
367                                         if (!numStr.isEmpty())
368                                             precioFinal = Double.parseDouble(numStr);
369                                     } catch (Exception e) {
370                                     }
371                                 }
372                             }
373                             p.setPrecio(precioFinal > 0 ? precioFinal : 1.99);
374                             mapaProductos.put(clave, p);
375                         }
376
377                         String categoria = mapearCategoria(nombre, categoriaCruda);
378                         if (!categoria.equals("General") && !categoria.isEmpty()) {
379                             categoria = categoria.substring(0, 1).toUpperCase() + categoria.substring(1);
380                         }
381                         p.setCategoria(categoria);
382                     } catch (Exception e) {
383                         logger.error("Error parseando línea: " + line);
384                     }
385                 }
386             }
387             if (!mapaProductos.isEmpty())
388                 repoProducto.saveAll(mapaProductos.values());
389             logger.info("Importados " + mapaProductos.size() + " de " + supermercado + ". Saltados: " + saltados);
390         } catch (Exception e) {
391             logger.error("Error leyendo CSV: " + e.getMessage());
392         }
393     }

```



4.4. Implementación del Frontend (Angular)

La interfaz de usuario de LinkedList se ha desarrollado siguiendo una arquitectura basada en componentes, lo que permite la reutilización de código y una mantenibilidad superior. Se ha buscado crear una *Single Page Application* (SPA) fluida y reactiva.

4.4.1. Estructura de Componentes y Navegación

La aplicación se divide en módulos lógicos que separan las áreas públicas (Login/Registro) de las áreas privadas de gestión. Cada vista principal cuenta con su propio componente de lógica (.ts), su plantilla de diseño (.html) y sus estilos específicos (.css).

- **Componente de Autenticación:** Gestiona los formularios de entrada y la lógica de guardado del Token JWT en el almacenamiento local (*Local Storage*).

```
export class AuthService {
  private apiUrl = 'https://localhost:8443/usuarios/login';
  private registerUrl = 'https://localhost:8443/usuarios/registro';
  // Observable para que otros componentes sepan cuando cambia el estado de la sesión
  public loginStatus$ = new BehaviorSubject<boolean>(this.isLoggedIn());
  constructor(private http: HttpClient, private router: Router) {}

  login(credentials: any) {
    // 1. Enviamos el POST con el JSON
    return this.http.post<any>(this.apiUrl, credentials).pipe(
      tap(res => {
        if (res && res.token) {
          this.saveToken(res.token); // Guardamos el JWT que nos da el servidor
          if (res.id) {
            localStorage.setItem('id', String(res.id));
          }
          if (res.nombre) {
            localStorage.setItem('nombre', res.nombre);
          }
          if (res.email) {
            localStorage.setItem('email', res.email);
          }
          this.loginStatus$.next(true); // Actualizamos el estado de la sesión
          this.router.navigate(['/perfil']); // Redirigimos al perfil después de login exitoso
        }
      })
    );
  }

  saveToken(token: string) {
    localStorage.setItem('token', token);
  }

  getToken(): string | null {
    return localStorage.getItem('token');
  }

  getId(): number | null {
    const id = localStorage.getItem('id');
    return id ? parseInt(id, 10) : null;
  }
}
```

- **Componente Dashboard:** Es el núcleo visual donde se renderizan las listas del usuario mediante el uso de directivas estructurales como *ngFor.

```
<div *ngFor="let pp of lista.productoPropios"
  class="list-group-item d-flex justify-content-between align-items-center p-3 border-0 mb-1"
  >
  <div class="d-flex align-items-center text-truncate flex-grow-1"
    style="min-width: 0;"
    >
    <input class="form-check-input me-3 shadow-xs cursor-pointer" type="checkbox"
      [checked]="pp.comprado"
      (change)="cambiarEstadoCompradoPropio(lista.codLista, pp)"
      [id]="'check-propio-' + lista.codLista + '-' + pp.id"
    >
    <label class="mb-0 fw-semibold cursor-pointer text-truncate"
      [ngClass]="{'text-decoration-line-through text-muted opacity-50': pp.comprado}"
      [for]="'check-propio-' + lista.codLista + '-' + pp.id"
      >
      {{ pp.nombre }}
    </label>
  </div>
```



- **Componente de Gestión de Productos:** Permite la interacción con el motor de scraping y la visualización de los detalles de cada artículo.

```
export class ProductoService {
  private baseUrl = 'https://localhost:8443/productos';
  private baseUrl2 = 'https://localhost:8443/productos- propios';

  constructor(private http: HttpClient, private authService: AuthService) { }

  private obtenerCabeceras(): HttpHeaders {
    const token = this.authService.getToken();
    return new HttpHeaders({
      'Content-Type': 'application/json',
      'Authorization': 'Bearer ${token}'
    });
  }

  listarProductos(): Observable<Producto[]> {
    return this.http.get<Producto[]>(this.baseUrl);
  }

  listarProductosPropiosDeUsuario(usuarioId: number): Observable<ProductoPropio[]> {
    return this.http.get<ProductoPropio[]>(`${this.baseUrl2}/${usuarioId}`, {
      headers: this.obtenerCabeceras()
    });
  }

  // Productos que el usuario ha subido al catálogo global (pendientes o confirmados)
  listarProductosCatalogSubidosPorUsuario(usuarioId: number): Observable<Producto[]> {
    const t = new Date().getTime();
    return this.http.get<Producto[]>(`${this.baseUrl2}/${usuarioId}/productos?t=${t}`, {
      headers: this.obtenerCabeceras()
    });
  }

  obtenerProductoPorId(id: number): Observable<Producto> {
    return this.http.get<Producto>(`${this.baseUrl}/${id}`);
  }
}
```

4.4.2. Servicios y Comunicación Asíncrona

Para que el Frontend pueda comunicarse con el servidor (Spring Boot), se han implementado Servicios de Angular. Estos servicios utilizan el módulo HttpClient para realizar peticiones asíncronas y gestionar las respuestas mediante Observables (RxJS).

```
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  constructor(private router: Router) { }

  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
    let cloned = req; // empieza con la petición original

    const token = localStorage.getItem('token');
    const isLoginOrRegister = req.url.includes('/login') || req.url.includes('/registrar');
    if (token && !isLoginOrRegister) {
      cloned = req.clone({
        headers: { Authorization: 'Bearer ${token}' }
      });
    }

    // --- Envía la petición al backend y captura errores ---
    return next.handle(cloned).pipe(
      catchError((error: HttpErrorResponse) => {
        if (error.status === 401) {
          console.warn('AuthInterceptor - 401 Unauthorized detected. Token expirado o inválido.');
```



4.4.3. Diseño Adaptativo y Experiencia de Usuario (UX)

Se ha integrado Bootstrap 5 para garantizar que la aplicación sea Responsive. Esto se ha implementado mediante el sistema de rejilla (*Grid System*), asegurando que elementos como las tarjetas de productos se reorganicen automáticamente según el tamaño de la pantalla del dispositivo (móvil, tablet o escritorio).

- **Validación de Formularios:** Se han utilizado formularios reactivos para ofrecer *feedback* en tiempo real al usuario (ej. avisar si un correo no tiene el formato correcto antes de enviar el formulario).

```
export class MensajeService {  
  
  private mensajeSubject = new BehaviorSubject<Mensaje | null>(null);  
  mensaje$ = this.mensajeSubject.asObservable();  
  
  private confirmacionSubject = new BehaviorSubject<Confirmacion | null>(null);  
  confirmacion$ = this.confirmacionSubject.asObservable();  
  
  constructor() { }  
  
  /* Muestra un mensaje de éxito que desaparece solo */  
  mostrarSuccess(texto: string) {  
    this.mostrar({ texto, tipo: 'success' });  
  }  
  mostrarInfo(texto: string) {  
    this.mostrar({ texto, tipo: 'info' });  
  }  
  /* Muestra un mensaje de error */  
  mostrarError(texto: string) {  
    this.mostrar({ texto, tipo: 'error' });  
  }  
  /* Abre un cuadro de diálogo para confirmar una acción */  
  confirmar(pregunta: string, alConfirmar: () => void) {  
    this.confirmacionSubject.next({ pregunta, callback: alConfirmar });  
  }  
  aceptarConfirmacion() {  
    const actual = this.confirmacionSubject.value;  
    if (actual) {  
      actual.callback();  
      this.cancelarConfirmacion();  
    }  
  }  
  cancelarConfirmacion() {  
    this.confirmacionSubject.next(null);  
  }  
}
```

- **Interceptores de Seguridad:** Se ha programado un interceptor que adjunta automáticamente el token JWT en la cabecera de cada petición, garantizando que el usuario siempre esté identificado sin intervención manual.



```

constructor(
  private http: HttpClient,
  private cd: ChangeDetectorRef,
  private authService: AuthService,
  private mensajeService: MensajeService,
  private productoService: ProductoService
) {
}

// Genera las cabeceras con el token JWT del usuario logueado para que el servidor nos deje hacer cosas protegidas.
private obtenerCabeceras(): HttpHeaders {
  const token = this.authService.getToken();

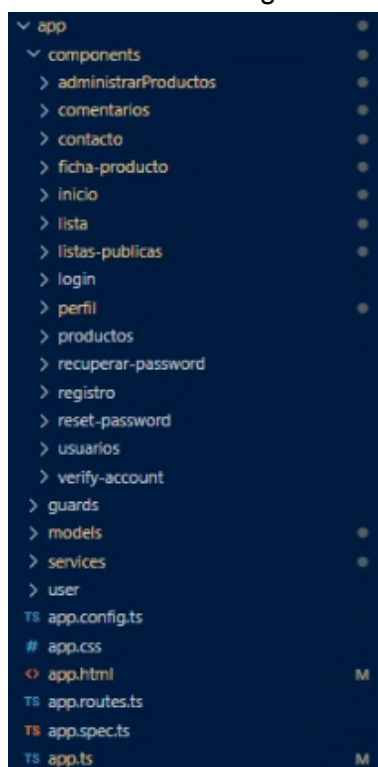
  return new HttpHeaders({
    'Content-Type': 'application/json',
    'Authorization': 'Bearer ${token}'
  });
}

// Al entrar mira si eres admin para decidir qué enseñarte.
ngOnInit(): void {
  this.authService.loginStatus$.subscribe(logged => {
    this.isLoggedIn = logged;
    this.isAdmin = this.authService.isAdmin();
    if (logged) {
      this.cargarListasDinamicas();
      this.cargarProductos();
    }
  });
}

```

4.4.4. Organización del Espacio de Trabajo

En el fronted nos organizamos con carpetas y subcarpetas y separacion de trabajo tal que asi:



4.5. Seguridad y Control de Versiones

4.5.1. Implementación de Seguridad con JWT (Stateless)

La seguridad del sistema se ha diseñado bajo un modelo Stateless. A diferencia de las sesiones tradicionales, el servidor no almacena información del usuario, lo que mejora la escalabilidad y rapidez.

El flujo de seguridad implementado sigue estos pasos:

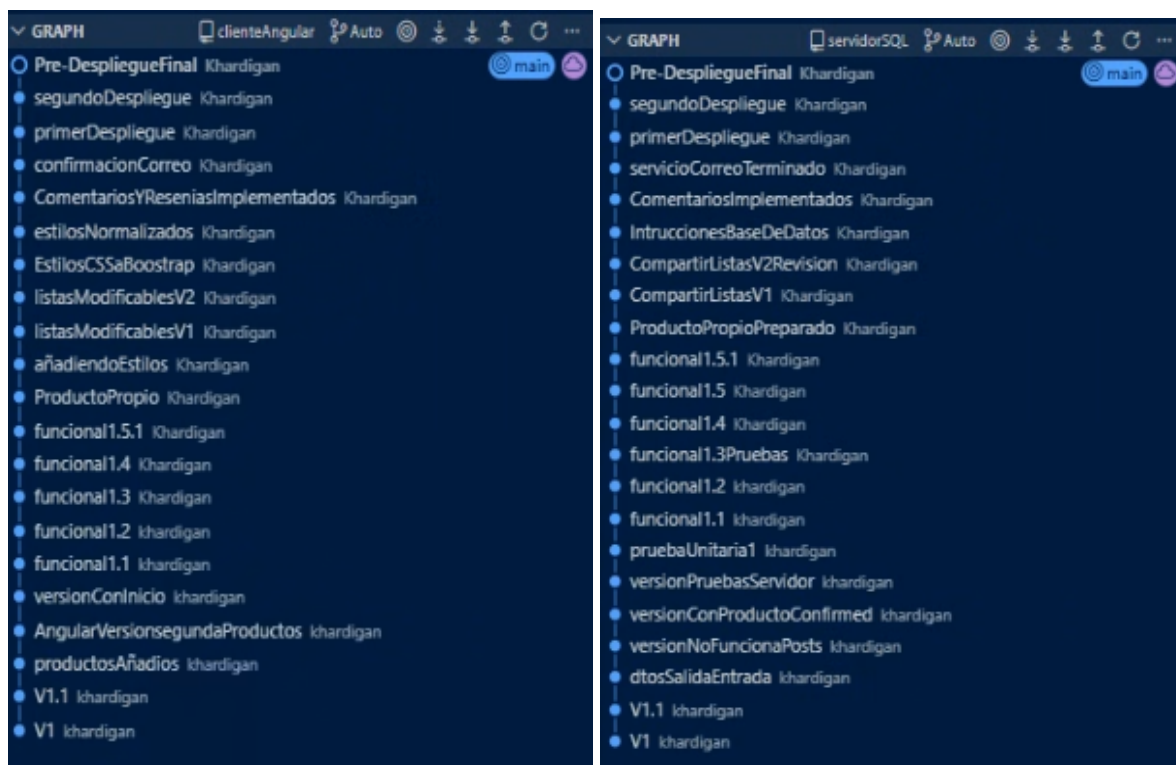
1. **Petición:** El cliente envía las credenciales.
2. **Emisión:** El servidor valida con **BCrypt** y firma un token JWT.
3. **Uso:** El cliente envía el token en la cabecera Authorization: Bearer.
4. **Validación:** El servidor verifica la firma y autoriza la petición.

```
public class serviceJWT {  
    private static final String SECRET = "esta_es_una_clave_super_secreta_de_ejemplo_1234567890";  
    private static final long EXPIRATION_TIME = 1000 * 60 * 60 * 24; // 24 horas  
    // Te da la clave secreta para firmar los tokens.  
    private SecretKey getSigningKey() {  
        return Keys.hmacShaKeyFor(SECRET.getBytes(StandardCharsets.UTF_8));  
    }  
    // Crea un token JWT para un usuario.  
    public String generarToken(String sub, String rol, Long id) {  
        Date ahora = new Date();  
        Date expiration = new Date(ahora.getTime() + EXPIRATION_TIME);  
  
        return Jwts.builder()  
            .setSubject(sub) // sub = nombre de usuario  
            .claim(name: "rol", rol) // rol = rol del usuario  
            .claim(name: "id", id) // <--- el ID del usuario  
            .setIssuedAt(ahora)  
            .setExpiration(expiration)  
            .signWith(getSigningKey(), SignatureAlgorithm.HS256)  
            .compact();  
    }  
  
    // Saca el nombre de usuario de dentro del token.  
    public String obtenerSubject(String token) {  
        return parseClaims(token).getBody().getSubject();  
    }  
  
    // Saca el rol del usuario de dentro del token.  
    public String obtenerRol(String token) {  
        return parseClaims(token).getBody().get(claimName: "rol", requiredType: String.class);  
    }  
  
    // Saca el ID del usuario de dentro del token.  
    public Long obtenerId(String token) {  
        Object idClaim = parseClaims(token).getBody().get("id");  
        if (idClaim == null)  
            return null;  
        if (idClaim instanceof Number num) {  
            return num.longValue();  
        }  
        return Long.valueOf(idClaim.toString());  
    }  
}
```



4.5.2. Control de versiones con GIT

Para la gestión del código fuente, se ha utilizado Git, permitiendo un desarrollo organizado y seguro. Se han separado los repositorios del Backend y del Frontend para mantener la independencia de los despliegues.



4.6. Despliegue y Virtualización (Docker)

Para cumplir con los requisitos de portabilidad, se ha implementado una solución mediante Docker. Esto garantiza que la aplicación funcione en cualquier equipo sin necesidad de instalar dependencias externas.

- **Aislamiento:** Cada servicio (Angular, Spring, Postgres) corre en un contenedor estanco, lo que evita conflictos entre versiones de software y librerías del sistema anfitrión.
- **Orquestación:** Se utiliza un archivo `docker-compose.yml` que levanta todo el ecosistema con un único comando, gestionando automáticamente la creación de redes internas para que los microservicios se comuniquen entre sí de forma segura.

Como usamos **Docker Hub**, se agiliza drásticamente el proceso de despliegue, ya que las imágenes personalizadas del Backend y el Frontend están pre-construidas y almacenadas en la nube. Esto permite que cualquier examinador o desarrollador externo pueda descargar la última versión estable del proyecto de manera inmediata, garantizando que el entorno de ejecución sea idéntico al de desarrollo.



Además, el uso de volúmenes de datos en Docker asegura que la persistencia de la base de datos PostgreSQL se mantenga incluso si los contenedores se detienen o se reinician, uniendo la flexibilidad de la virtualización con la seguridad de un sistema de archivos persistente.

```
docker-compose.yml
1  services:
2    db:
3      image: postgres:alpine
4      container_name: postgres_db
5      environment:
6        POSTGRES_DB: servidor_database
7        POSTGRES_PASSWORD: jose
8        POSTGRES_USER: postgres
9      healthcheck:
10       test: ["CMD-SHELL", "pg_isready -U postgres -d servidor_database"]
11       interval: 5s
12       timeout: 5s
13       retries: 5
14     networks:
15       - red-proyecto
16     ports:
17       - "5432:5432"
18     restart: unless-stopped
19
20   servidor:
21     image: khardigan/proyecto-servidor:v1
22     container_name: spring_backend
23     ports:
24       - "8443:8443"
25     depends_on:
26       db:
27         condition: service_healthy
28     environment:
29       - SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/servidor_database
30       - SPRING_DATASOURCE_USERNAME=postgres
31       - SPRING_DATASOURCE_PASSWORD=jose
32       - SPRING_JPA_HIBERNATE_DDL_AUTO=update
33     networks:
34       - red-proyecto
35     restart: unless-stopped
36
37   cliente:
38     image: khardigan/proyecto-frontend:v1
39     container_name: angular_frontend
40     ports:
41       - "4200:80"
42     depends_on:
43       - servidor
44     networks:
45       - red-proyecto
46     restart: unless-stopped
47
48   networks:
49     red-proyecto:
50       driver: bridge
```



5. PRUEBAS

Herramientas utilizadas:

Para la implementación de las pruebas se han utilizado las siguientes herramientas:

- **JUnit 5** – desarrollo de pruebas unitarias, de integración y de carga
- **Mockito** – simulación de dependencias en pruebas unitarias.
- **SpringBootTest y MockMvc** – ejecución de pruebas de integración sobre endpoints REST.
- **AtomicInteger y ExecutorService** – para simulación de carga y concurrencia.

5.1. Pruebas de módulos (Unitarias)

En esta fase se ha verificado el funcionamiento aislado de cada componente del Backend.

Archivo :

Clase: ServiceAuthenTest

Tipo: **Pruebas Unitarias con Mockito**

Objetivo: El objetivo de estas pruebas es verificar el correcto funcionamiento de la lógica de negocio del servicio de autenticación de forma aislada, mediante la simulación de sus dependencias externas.

5.1.1 guardarUsuario_ok_cuandoNoExiste

Tipo de prueba: Prueba unitaria positiva

Objetivo: Se comprueba que el usuario no se encuentra en el repositorio, se ejecuta la operación de guardado y se valida que el objeto resultante es correcto, que sus atributos han sido inicializados adecuadamente y que el repositorio ha sido invocado..

Validaciones realizadas:

- Se comprueba que el usuario no exista en el repositorio, se ejecuta el guardado del usuario y se verifica que:
 - El resultado no sea nulo.
 - El nombre sea correcto.
 - El perfil del usuario sea inicializado.
 - La lista de productos subidos sea inicializada.
 - El método save() del repositorio haya sido invocado.

Resultado esperado:

El usuario se guarda correctamente en el sistema.



```

43  @Test
44  void guardarUsuario_ok_cuandoNoExiste() {
45      Usuario usuario = new Usuario(
46          nombre: "pepe",
47          email: "pepe@email.com",
48          contraseña: "1234",
49          rol: "USER",
50          LocalDate.now());
51
52      when(repoUsuario.findAll()).thenReturn(List.of());
53      when(repoUsuario.save(any(type: Usuario.class))).thenReturn(i -> i.getArgument(index: 0));
54
55      Usuario resultado = serviceAuthen.guardarUsuario(usuario);
56
57      assertNotNull(resultado);
58      assertEquals("pepe", resultado.getNombre());
59      assertNotNull(resultado.getPerfilUsuario());
60      assertNotNull(resultado.getListaProductosSubidos());
61
62      verify(repoUsuario).save(usuario);
63  }
64
65  @Test
66  void guardarUsuario_lanzaExcepcion_siUsuarioExiste() {
67      Usuario usuario = new Usuario();
68      usuario.setNombre(nombre: "admin");

```

PROBLEMS 45 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

```

%TESTC 1 v2
%TSTTREE2,com.example.demo.ServiceAuthenTest,true,1,false,1,ServiceAuthenTest,,[engine:junit-jupiter]/[class:com.example.demo.ServiceAuthenTest]
%TSTTREE3,guardarUsuario_ok_cuandoNoExiste(com.example.demo.ServiceAuthenTest),false,1,false,2,guardarUsuario_ok_cuandoNoExiste(),,[engine:junit-jupiter]/[class:com.
example.demo.ServiceAuthenTest]/[method:guardarUsuario_ok_cuandoNoExiste()]
%TESTS 3,guardarUsuario_ok_cuandoNoExiste(com.example.demo.ServiceAuthenTest)

%TESTE 3,guardarUsuario_ok_cuandoNoExiste(com.example.demo.ServiceAuthenTest)

%RUNTIME1135

```

Test Runner for Java

- ServiceAuthenTest \$(symbol-namespace) com.example.demo
- guardarUsuario_ok_cuandoNoExiste() \$(symbol-class) ServiceAuthenTest

5.1.2 guardarUsuario_lanzaExcepcion_siUsuarioExiste

Tipo de prueba: Prueba unitaria negativa

Objetivo: Se simula la existencia previa del usuario y se valida que se lanza la excepción correspondiente, así como que no se realiza la operación de guardado. .

Validaciones realizadas:

- Se simula que el usuario ya existe en el repositorio, se ejecuta el método de guardado y se verifica que:
 - Se lance la excepción `RecursoDuplicadoException`.
 - El método `save()` no sea ejecutado.

Resultado esperado:

Se lanza la excepción correspondiente y el usuario no se guarda.

```

64
65  @Test
66  void guardarUsuario_lanzaExcepcion_siUsuarioExiste() {
67      Usuario usuario = new Usuario();
68      usuario.setNombre(nombre: "admin");
69
70      Usuario existente = new Usuario();
71      existente.setNombre(nombre: "admin");
72
73      when(repoUsuario.findAll()).thenReturn(List.of(existente));
74
75      assertThrows(
76          RecursoDuplicadoException.class,
77          () -> serviceAuthen.guardarUsuario(usuario));
78
79      verify(repoUsuario, never()).save(any());
80  }
81

```

PROBLEMS 45 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

Test Runner for Java

- ServiceAuthenTest \$(symbol-namespace) com.example.demo
- guardarUsuario_lanzaExcepcion_siUsuarioExiste() \$(symbol-class) ServiceAuthenTest

> 1 older result



5.1.3 guardarUsuario_debeCifrarPassword

Tipo de prueba: Prueba unitaria de seguridad

Objetivo: Se simula el comportamiento del codificador de contraseñas y se verifica que el valor almacenado corresponde al resultado del cifrado.

Validaciones realizadas:

- Se simula el cifrado mediante PasswordEncoder, se ejecuta el guardado del usuario y se verifica que:
 - La contraseña no sea nula.
 - La contraseña almacenada coincida con el valor cifrado.
 - El método encode() haya sido llamado.

Resultado esperado:

La contraseña se almacena correctamente cifrada.

```
82  @Test
83  public void guardarUsuario_debeCifrarPassword() {
84      Usuario usuario = new Usuario(nombre: "ana", email: "ana@mail.com", contraseña: "password123", rol: "USER", LocalDate.now());
85      when(repoUsuario.findAll()).thenReturn(List.of());
86      when(passwordEncoder.encode(anyString())).thenReturn(value: "bcrypted_password");
87      when(passwordEncoder.matches(anyString(), anyString())).thenReturn(value: true);
88
89      when(repoUsuario.save(any(type: Usuario.class))).thenAnswer(i -> i.getArgument(index: 0));
90
91      Usuario resultado = serviceAuthen.guardarUsuario(usuario);
92
93      assertNotNull(resultado.getContraseña());
94      assertTrue(passwordEncoder.matches("password123", resultado.getContraseña()));
95      verify(passwordEncoder).encode("password123");
96  }
97
```

PROBLEMS 45 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

Test Runner for Java

ServiceAuthenTest \$(symbol-namespace) com.example.demo - \$(project) demo

guardarUsuario_debeCifrarPassword() \$(symbol-class) ServiceAuthenTest - \$(symbol-namespace) com.e...

> 2 older results

%TESTC 1 v2

%STTREE2,com.example.demo.ServiceAuthenTest,true,1,f

%STTREE3,guardarUsuario_debeCifrarPassword(com.examp

m.example.demo.ServiceAuthenTest)[method:guardarUsua

%TESTS 3,guardarUsuario_debeCifrarPassword(com.examp

%TESTE 3,guardarUsuario_debeCifrarPassword(com.examp

5.1.4 login_exitoso_devuelveToken

Tipo de prueba: Prueba unitaria positiva (login)

Objetivo: Se simula un usuario válido, una contraseña correcta y la generación de un token JWT, verificando que la respuesta contiene los datos esperados.



Validaciones realizadas:

- Se simula un usuario existente, se simula coincidencia de contraseña, se simula generación de token JWT y se verifica que:
 - La respuesta no sea nula.
 - El token generado sea correcto.
 - El nombre del usuario sea correcto.

Resultado esperado:

Se devuelve un AuthResponse con el token JWT.

```
98  @test
99  void login_exitoso_devuelveToken() {
100      Usuario usuario = new Usuario();
101      usuario.setNombre(nombre: "juan");
102      usuario.setContraseña(contraseña: "encodedPassword");
103      usuario.setRol(rol: "USER");
104      usuario.setId(id: 1L);
105
106      when(repoUsuario.findAll()).thenReturn(List.of(usuario));
107      when(passwordEncoder.matches("password", "encodedPassword")).thenReturn(value: true);
108      when(jwtService.generarToken(sub: "juan", rol: "USER", id: 1L)).thenReturn(value: "token_jwt");
109
110      com.example.demo.proyecto.dto.AuthResponse response = serviceAuthen.login(nombre: "juan", password: "password");
111
112      assertNotNull(response);
113      assertEquals("token_jwt", response.getToken());
114      assertEquals("juan", response.getNombre());
115  }
116
```

Test Runner for Java

ServiceAuthenTest \$(symbol-namespace) com.example.demo + \$(project) demo

login_exitoso_devuelveToken() \$(symbol-class) ServiceAuthenTest + \$(symbol-namespace) ...

3 older results

%TESTC 1 v2
%TSTTREE2,com.example.demo.ServiceAuthenTest
%TSTTREE3,login_exitoso_devuelveToken(com.e
mo.ServiceAuthenTest)/[method:login_exitoso
%TESTS 3,login_exitoso_devuelveToken(com.e

5.1.5 login_fallido_siPasswordIncorrecta

Tipo de prueba: Prueba unitaria negativa (login)

Objetivo: Se simula una contraseña incorrecta y se comprueba que el sistema rechaza el acceso.

Validaciones realizadas:

- Se simula usuario existente, se simula contraseña incorrecta, se ejecuta login y se verifica que el resultado sea nulo.

Resultado esperado:

El login retorna null.



```

117 @Test
118 void login_fallido_siPasswordIncorrecta() {
119     Usuario usuario = new Usuario();
120     usuario.setNombre(nombre: "juan");
121     usuario.setContraseña(contraseña: "encodedPassword");
122
123     when(repoUsuario.findAll()).thenReturn(List.of(usuario));
124     when(passwordEncoder.matches("wrongpass", "encodedPassword")).thenReturn(value: false);
125
126     com.example.demo.proyecto.dto.AuthResponse response = serviceAuthen.login(nombre: "juan", password: "wrongpass");
127
128     assertNull(response);
129 }

```

PROBLEMS 45 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

Test Runner for Java

ServiceAuthenTest \$(symbol-namespace) com.example.demo - \$(project) demo

login_fallido_siPasswordIncorrecta() \$(symbol-class) ServiceAuthenTest - \$(symbol-namespace) com.exa...

> 4 older results

```

%TESTC 1 v2
%TSTTREE2,com.example.demo.ServiceAuthenTest
%TSTTREE3,login_fallido_siPasswordIncorrecta
com.example.demo.ServiceAuthenTest[/method
%TESTS 3,login_fallido_siPasswordIncorrecta

%TESTE 3,login_fallido_siPasswordIncorrecta

%RUNTIME906

```

5.1.6 actualizarUsuario_debeGuardarCambios

Tipo de prueba: Prueba unitaria de actualización

Objetivo: Se verifica que los cambios introducidos se aplican correctamente y que son persistidos en el repositorio.

Validaciones realizadas:

- Se envían nuevos datos, se ejecuta el método de actualización y se verifica que:
 - El resultado no sea nulo.
 - El ID sea correcto.
 - El nombre haya sido actualizado.
 - El repositorio haya guardado los cambios.

Resultado esperado:

El usuario se actualiza correctamente.

```

130 @Test
131 void actualizarUsuario_debeGuardarCambios() {
132     Usuario datosNuevos = new Usuario();
133     datosNuevos.setNombre(nombre: "Juan Modificado");
134
135     when(repoUsuario.save(any(type: Usuario.class))).thenReturn(i -> i.getArgument(index: 0));
136
137     Usuario resultado = serviceAuthen.actualizarUsuario(id: 1L, datosNuevos);
138
139     assertNotNull(resultado);
140     assertEquals(1L, resultado.getId());
141     assertEquals("Juan Modificado", resultado.getNombre());
142     verify(repoUsuario).save(datosNuevos);
143 }
144
145
146
147

```

PROBLEMS 45 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

Test Runner for Java

ServiceAuthenTest \$(symbol-namespace) com.example.demo - \$(project) demo

actualizarUsuario_debeGuardarCambios() \$(symbol-class) ServiceAuthenTest - \$(symbol-namespace) com...

> 5 older results

```

%TESTC 1 v2
%TSTTREE2,com.examp
%TSTTREE3,actualiza
ass:com.example.dem
%TESTS 3,actualiza

%TESTE 3,actualiza

```



5.2.Pruebas de Integración

Clase: AuthenIntegrationTest

Tipo: **Pruebas de Integración con SpringBootTest y MockMvc**

Objetivo: stas pruebas tienen como objetivo validar el funcionamiento conjunto de los distintos componentes del sistema (controlador, servicio y repositorio).

5.2.1 registrarUsuario_integracion_ok

Tipo de prueba: Integración positiva

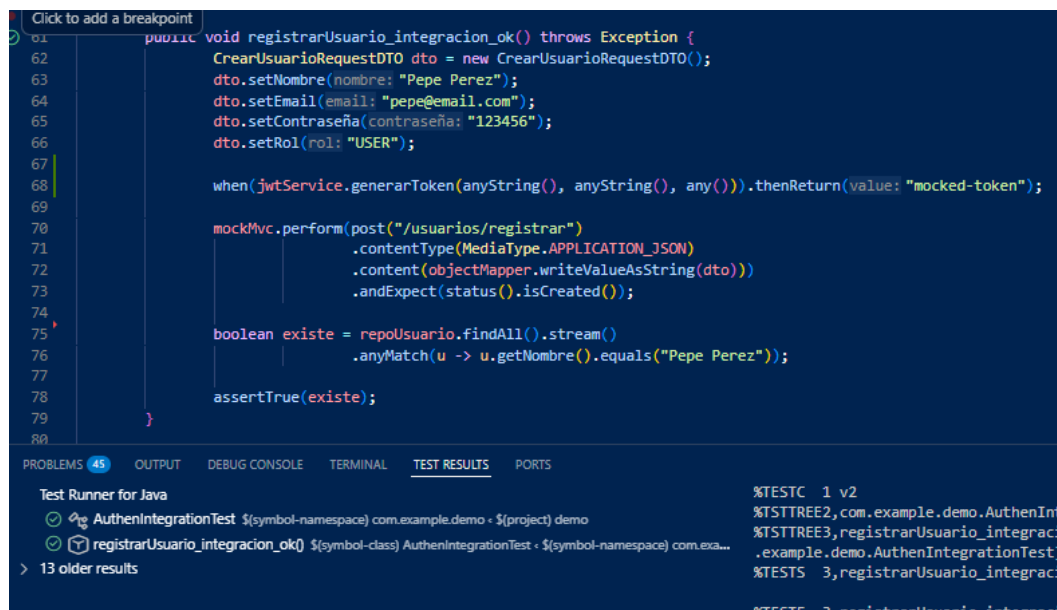
Objetivo: Verificar que el endpoint /usuarios/registrar crea correctamente un usuario en el sistema.

Validaciones realizadas:

- Se envía petición POST con datos válidos, se espera código HTTP 201 (Created). y se verifica directamente en base de datos que el usuario exista.

Resultado esperado:

Usuario creado y persistido correctamente.



```
61 public void registrarUsuario_integracion_ok() throws Exception {
62     CrearUsuarioRequestDTO dto = new CrearUsuarioRequestDTO();
63     dto.setNombre(nombre: "Pepe Perez");
64     dto.setEmail(email: "pepe@email.com");
65     dto.setContraseña(contraseña: "123456");
66     dto.setRol(rol: "USER");
67
68     when(jwtService.generarToken(anyString(), anyString(), any())).thenReturn(value: "mocked-token");
69
70     mockMvc.perform(post("/usuarios/registrar")
71         .contentType(MediaType.APPLICATION_JSON)
72         .content(objectMapper.writeValueAsString(dto))
73         .andExpect(status().isCreated());
74
75     boolean existe = repoUsuario.findAll().stream()
76         .anyMatch(u -> u.getNombre().equals("Pepe Perez"));
77
78     assertTrue(existe);
79 }
80
```

Test Runner for Java

- ✓ AuthenIntegrationTest \$(symbol-namespace) com.example.demo - \$(project) demo
- ✓ registrarUsuario_integracion_ok() \$(symbol-class) AuthenIntegrationTest - \$(symbol-namespace) com.exa...

> 13 older results

```
%TESTC 1 v2
%TSTTREE2,com.example.demo.AuthenInt
%TSTTREE3,registrarUsuario_integraci
.example.demo.AuthenIntegrationTest]
%TESTS 3,registrarUsuario_integraci
%TESTE 3 registrarUsuario_integraci
```

5.2.2 contextLoads

Tipo de prueba: Prueba de carga de contexto

Objetivo: Comprueba que el contexto de la aplicación se carga correctamente.



Validaciones realizadas:

- Se verifica que MockMvc no sea nulo y se verifica que los repositorios estén correctamente inyectados.

Resultado esperado:

El contexto de la aplicación se carga sin errores.

```
85     @Test
86     public void contextLoads() {
87         // Verifica que el contexto de Spring se cargue correctamente
88         // y que los beans principales estén presentes.
89         org.junit.jupiter.api.Assertions.assertNotNull(mockMvc);
90         org.junit.jupiter.api.Assertions.assertNotNull(repoUsuario);
91     }
92
```

PROBLEMS 45 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

Test Runner for Java

- ✓ AuthenIntegrationTest \$(symbol-namespace) com.example.demo - \$(project) demo
- ✓ contextLoads() \$(symbol-class) AuthenIntegrationTest - \$(symbol-namespace) com.example.demo - \$(proje...

> 14 older results

%TESTC 1 v2
%TSTTREE2, com.
%TSTTREE3, cont
method: context
%TESTS 3, cont
%TSTTE 3, cont

5.2.3 listarUsuarios_integracion_debeRetornarLista

Tipo de prueba: Integración de endpoint GET

Objetivo: Verifica el correcto funcionamiento del endpoint de consulta de usuarios.

Validaciones realizadas:

- Se realiza petición GET y se espera código HTTP 200 (OK)

Resultado esperado:

El endpoint responde correctamente.

```
93     @Test
94     public void listarUsuarios_integracion_debeRetornarLista() throws Exception {
95         // Simulamos un token válido para acceder al endpoint protegido
96         // Nota: En AuthenIntegrationTest ya está @AutoConfigureMockMvc(addFilters =
97         // false)
98         mockMvc.perform(org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get("/usuarios")
99             .accept(MediaType.APPLICATION_JSON))
100             .andExpect(status().isOk());
101     }
102
```

PROBLEMS 45 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

Test Runner for Java

- ✓ AuthenIntegrationTest \$(symbol-namespace) com.example.demo - \$(project) demo
- ✓ listarUsuarios_integracion_debeRetornarLista() \$(symbol-class) AuthenIntegrationTest - \$(symbol-names...

> 15 older results

%TESTC 1 v2
%TSTTREE2, com.example.demo.AuthenInteg
%TSTTREE3, listarUsuarios_integracion_d
:junit-jupiter]/[class:com.example.dem
%TESTS 3, listarUsuarios_integracion_d
%TSTTE 3, listarUsuarios_integracion_d



5.2.4 login_integracion_fallido_credencialesMalas

Tipo de prueba: Integración negativa

Objetivo: Verifica que el sistema rechaza credenciales incorrectas en el proceso de autenticación.

Validaciones realizadas:

- Se envía petición POST con usuario erróneo y se espera código HTTP 401 (Unauthorized).

Resultado esperado:

El sistema rechaza el acceso correctamente.

```
102
103
104  @Test
105  public void login_integracion_fallido_credencialesMalas() throws Exception {
106      // Creamos un usuario válido en la base de datos con contraseña cifrada
107      Usuario u = new Usuario();
108      u.setNombre(nombre: "usuario_valido");
109      u.setContraseña(passwordEncoder.encode("123456"));
110      u.setRol(rol: "USER");
111      u.setEmail(email: "usuario@correo.com");
112      u.setActivo(activo: true);
113      repoUsuario.save(u);
114
115      // Intentamos login con contraseña incorrecta
116      String jsonBody = "{\"nombre\":\"usuario_valido\", \"password\":\"mal\"}";
117      mockMvc.perform(post("/usuarios/login")
118                  .contentType(MediaType.APPLICATION_JSON)
119                  .content(jsonBody)
120                  .andExpect(status().isUnauthorized());
121  }
122 }
```

PROBLEMS 45 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

Test Runner for Java

- ✓ AuthenIntegrationTest \$(symbol-namespace) com.example.demo - \$(project) demo
- ✓ login_integracion_fallido_credencialesMalas() \$(symbol-class) AuthenIntegrationTest - \$(symbol-namesp...

> 16 older results

```
%TESTC 1 v2
%TSTTREE2,com.example.demo.A
%TSTTREE3,login_integracion_
unit-jupiter]/[class:com.exa
%TESTS 3,login_integracion_
%TESTE 3,login_integracion_
%RUNTIME5388
```

5.3. Pruebas de Carga y Rendimiento

Archivo: LoadSimulationTest.java

Tipo: Pruebas de carga, rendimiento y estrés simuladas con JUnit

Objetivo: Evaluar el comportamiento del sistema bajo concurrencia, medir tiempos de ejecución y comprobar la estabilidad ante operaciones repetitivas.



5.3.1 Prueba de Concurrencia

- **Objetivo:** Simular 50 usuarios (hilos) ejecutando operaciones al mismo tiempo para verificar que el sistema maneja correctamente la concurrencia.
- **Método:** `testSimulacionConcurrenciaComentarios()`
- **Validaciones realizadas:**
 - Se crean 50 hilos simultáneos.
 - Cada hilo realiza una operación simulada y aumenta un contador si termina correctamente.
 - Se comprueba que todos los hilos terminen dentro de 5 segundos.
 - Se verifica que todas las operaciones se hayan ejecutado correctamente.
- **Resultado esperado:**
 - Todas las tareas terminan correctamente sin bloqueos ni errores

```
82  @Test
83  void testSimulacionConcurrenciaComentarios() throws InterruptedException {
84      when(repoComentario.findAll()).thenReturn(mockComentarios);
85
86      int numeroDeHilos = 50;
87      ExecutorService executor = Executors.newFixedThreadPool(numeroDeHilos);
88      AtomicInteger contadorExitos = new AtomicInteger(0);
89
90      Runnable tarea = () -> {
91          try {
92              List<ComentarioDTO> resultado = comentarioService.obtenerTodosComentarios();
93              if (resultado != null && resultado.size() == 1000) {
94                  contadorExitos.incrementAndGet();
95              }
96          } catch (Exception e) {
97              e.printStackTrace();
98          }
99      };
100
101      for (int i = 0; i < numeroDeHilos; i++) {
102          executor.submit(tarea);
103      }
104
105      executor.shutdown();
106      boolean terminado = executor.awaitTermination(5, TimeUnit.SECONDS);
107
108      assertTrue(terminado, message: "Todas las tareas deberían haber terminado");
109      assertTrue(contadorExitos.get() == numeroDeHilos, message: "Todas las tareas deberían haber sido exitosas");
110      System.out.println("Prueba de Concurrencia: 50 hilos obtuvieron los comentarios correctamente.");
111  }
112
113  /*
```

PROBLEMS 45 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

Test Runner for Java

- ✓ LoadSimulationTest \$(symbol-namespace) com.example.demo - \$(project) demo
- ✓ testSimulacionConcurrenciaComentarios() \$(symbol-class) LoadSimulationTest - \$(symbol-namespace) c...
- > 17 older results

%TESTC 1 v2
%TSTTREE2,com.example.demo.LoadSimulat
%TSTTREE3,testSimulacionConcurrenciaCor
class:com.example.demo.LoadSimulationT
%TESTS 3,testSimulacionConcurrenciaCor
%TESTE 3 testSimulacionConcurrenciaCor



5.3.2 Prueba de Rendimiento

- **Objetivo:** Se mide el tiempo de ejecución de una operación concreta.
- **Método:** testRendimientoObtencionComentarios()
- **Validaciones realizadas:**
 - Se registra el tiempo de inicio y fin de la operación.
 - Se comprueba que la operación completa no tarde más de 500ms.
- **Resultado esperado:**
 - La operación se ejecuta en menos de 500ms.

```
120  @Test
121  void testRendimientoObtencionComentarios() {
122      when(repoComentario.findAll()).thenReturn(mockComentarios);
123
124      long inicio = System.currentTimeMillis();
125
126      List<ComentarioDTO> resultado = comentarioService.obtenerTodosComentarios();
127
128      long fin = System.currentTimeMillis();
129      long duracion = fin - inicio;
130
131      System.out.println("Duración de mapear 1000 comentarios: " + duracion + " ms");
132
133      assertTrue(resultado.size() == 1000);
134      assertTrue(duracion < 1000, message: "El sistema no debería tardar más de 1000ms en procesar 1000 comentarios");
135  }
```

PROBLEMS 45 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

Test Runner for Java

- ✓ LoadSimulationTest \$(symbol-namespace) com.example.demo < \$(project) demo
- ✓ testRendimientoObtencionComentarios() \$(symbol-class) LoadSimulationTest < \$(symbol-namespace) co...

> 18 older results

```
%TESTC 1 v2
%STTREE2,com.example.demo.LoadSimulationT
%STTREE3,testRendimientoObtencionComentar
s:com.example.demo.LoadSimulationTest]/[me
%TESTS 3,testRendimientoObtencionComentar
```

5.3.3 Prueba de Estrés

- **Objetivo:** Ejecutar una operación sencilla muchas veces consecutivas para comprobar la estabilidad del sistema bajo carga repetitiva.
- **Método:** testEstresBucle()
- **Validaciones realizadas:**
 - Se ejecuta un bucle de 10.000 iteraciones realizando una operación matemática simple.



- Se mide el tiempo total de ejecución.
- **Resultado esperado:**
 - La operación se completa sin errores ni interrupciones, demostrando resistencia ante carga intensa.

```

144  @Test
145  void testEstresProcesamientoComentarios() {
146      when(repoComentario.findAll()).thenReturn(mockComentarios);
147
148      int iteraciones = 10000;
149      long inicio = System.currentTimeMillis();
150
151      for (int i = 0; i < iteraciones; i++) {
152          comentarioService.obtenerTodosComentarios();
153      }
154
155      long fin = System.currentTimeMillis();
156
157      System.out.println("Prueba de Estrés: " + iteraciones + " peticiones procesadas en " + (fin - inicio) + " ms");
158
159      assertTrue(condition: true);
160  }
161  }

```

PROBLEMS 45 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

Test Runner for Java

- LoadSimulationTest \$(symbol-namespace) com.example.demo - \$(project) demo
- testEstresProcesamientoComentarios() \$(symbol-class) LoadSimulationTest - \$(symbol-namespace) com...

> 19 older results

```

%TESTC 1 v2
%TSTTREE2,com.example.demo.LoadSimulationTe
%TSTTREE3,testEstresProcesamientoComentari
com.example.demo.LoadSimulationTest]/[metho
%TESTS 3,testEstresProcesamientoComentari
%TESTE 3,testEstresProcesamientoComentari
%RUNTIME1224

```

6. SEGURIDAD

6.1. Mejoras de seguridad aplicadas

La seguridad se ha implementado de forma transversal en todo el proyecto, aplicando el principio de "defensa en profundidad". A continuación, se detallan los puntos concretos solicitados en las revisiones anteriores:

6.1.1. Cifrado de credenciales con BCrypt

Para cumplir con la LOPD y asegurar que las contraseñas no sean legibles ni siquiera por los administradores de la base de datos, se ha implementado el algoritmo de hashing BCrypt.



```
// Encriptación de contraseñas
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

// Configuración CORS para Angular
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOriginPatterns(List.of("*"));
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "PATCH", "OPTIONS"));
    config.setAllowedHeaders(List.of("*"));
    config.setAllowCredentials(true);

    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", config);
    return source;
}
```

6.1.2. Gestión de Autorización mediante JWT

Se ha implementado un sistema de seguridad Stateless basado en tokens. Esto evita el uso de sesiones en el servidor, mejorando el rendimiento y la escalabilidad de la aplicación.

- **Generación del Token:** Al hacer login, el servidor genera una firma digital que incluye los roles del usuario.
- **Interceptor de Seguridad:** Se ha configurado un filtro (OncePerRequestFilter) que intercepta cada petición entrante para validar la autenticidad del token antes de permitir el acceso al controlador.

```
public class serviceJWT {
    private static final String SECRET = "esta_es_una_clave_super_secreta_de_ejemplo_1234567890";
    private static final long EXPIRATION_TIME = 1000 * 60 * 60 * 24; // 24 horas
    // Te da la clave secreta para firmar los tokens.
    private SecretKey getSigningKey() {
        return Keys.hmacShaKeyFor(SECRET.getBytes(StandardCharsets.UTF_8));
    }
    // Crea un token JWT para un usuario.
    public String generarToken(String sub, String rol, Long id) {
        Date ahora = new Date();
        Date expiration = new Date(ahora.getTime() + EXPIRATION_TIME);

        return Jwts.builder()
            .setSubject(sub) // sub = nombre de usuario
            .claim(name: "rol", rol) // rol = rol del usuario
            .claim(name: "id", id) // ---- el ID del usuario
            .setIssuedAt(ahora)
            .setExpiration(expiration)
            .signWith(getSigningKey(), SignatureAlgorithm.HS256)
            .compact();
    }

    // Saca el nombre de usuario de dentro del token.
    public String obtenerSubject(String token) {
        return parseClaims(token).getBody().getSubject();
    }

    // Saca el rol del usuario de dentro del token.
    public String obtenerRol(String token) {
        return parseClaims(token).getBody().get("claimName: rol", requiredType: String.class);
    }
}
```



6.1.3. Configuración de Políticas CORS y Permisos de Rutas

Uno de los puntos críticos fue permitir la comunicación entre el puerto de Angular (4200) y el de Spring Boot (8080). Se ha configurado una política de **CORS (Cross-Origin Resource Sharing)** específica para restringir el acceso solo a orígenes de confianza.

Además, se han definido los permisos de las rutas de la API, diferenciando claramente qué recursos son públicos y cuáles requieren autenticación Bearer.

```
public class SecurityConfig {  
  
    private final JwtAuthenticationFilter jwtAuthFilter;  
  
    public SecurityConfig(JwtAuthenticationFilter jwtAuthFilter) {  
        this.jwtAuthFilter = jwtAuthFilter;  
    }  
  
    @Bean  
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {  
        http  
            .csrf(csrf -> csrf.disable())  
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))  
            .sessionManagement(session -> session.sessionCreationPolicy(  
                org.springframework.security.config.http.SessionCreationPolicy.STATELESS))  
            .authorizeHttpRequests(auth -> auth  
                .requestMatchers("/", "/usuarios/login", "/usuarios/registrarse").permitAll()  
                .requestMatchers("/productos", "/productos/buscar", "/productos/categorias",  
                    "/productos/supermercados", "/productos/pending")  
                .permitAll()  
                .requestMatchers(HttpMethod.GET, "/comentarios/**").permitAll()  
                .requestMatchers(HttpMethod.GET, "/usuarios/comentarios/**").permitAll()  
                .requestMatchers("/listas/**").permitAll()  
                .requestMatchers("/public/**").permitAll()  
                .anyRequest().authenticated()  
            )  
            .addFilterBefore(jwtAuthFilter,  
                org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter.class);  
  
        return http.build();  
    }  
}
```

7. CONCLUSIONES

7.1. Grado de consecución de los objetivos inicialmente planteados

El proyecto comenzó como una propuesta orientada a resolver las dificultades de organización en las listas de la compra tradicionales, buscando automatizar parte del proceso de búsqueda de productos y comparación de precios mediante tecnologías web modernas.

Durante el desarrollo se ha conseguido construir una aplicación funcional completa, compuesta por un Backend en Spring Boot y un Frontend en Angular, desplegada mediante Docker y preparada para funcionar de forma portable en distintos entornos.



Además, se ha implementado un sistema de scraping capaz de extraer información real de supermercados online, aportando valor añadido respecto a una lista de compra convencional.

Tras la finalización del periodo de desarrollo, se puede afirmar que el grado de consecución de los objetivos es muy satisfactorio:

- **Integración de Tecnologías:** Se ha logrado unificar de forma eficiente un Backend robusto en Spring Boot 3 con un Frontend dinámico en Angular 17, demostrando la viabilidad de la arquitectura de microservicios.
- **Automatización de Datos:** El motor de Web Scraping es capaz de interactuar con el DOM de sitios externos y extraer información útil, cumpliendo con el objetivo de reducir la carga manual de datos para el usuario (actualmente el sistema de scraping se encuentra adaptado para dos supermercados, aunque la arquitectura desarrollada permite ampliar el soporte a nuevos proveedores de forma escalable).
- **Seguridad y Privacidad:** Se ha implementado un sistema de autenticación JWT que garantiza que la información de las listas de la compra es privada y solo accesible por su propietario.
- **Despliegue y Portabilidad:** Gracias a la contenedorización con Docker, se ha cumplido el objetivo de que el proyecto sea autoinstalable y ejecutable en cualquier entorno sin dependencias externas.

7.2. Dificultades encontradas

7.2.1. Técnicas y de otro tipo

Dificultades Técnicas:

1. **Gestión de CORS y Seguridad:** Uno de los mayores retos fue la configuración de la política de intercambio de recursos de origen cruzado (CORS). Al tener el frontend y el backend en contenedores distintos, la comunicación inicial era bloqueada por el navegador, lo que requirió una configuración profunda en el SecurityFilterChain de Spring.
2. **Dinamismo del HTML en el Scraping:** El scraping se encontró con la dificultad de que cada tienda online estructura su HTML de forma distinta y, en ocasiones, de forma dinámica mediante JavaScript. Se tuvo que adaptar al scraping de cada supermercado diferente para hacer la extracción más flexible y tolerante a fallos.
3. **Persistencia en Docker:** Configurar los volúmenes de PostgreSQL en Docker para que los datos no se borraran al apagar los contenedores requirió una curva de aprendizaje específica sobre la gestión de volúmenes persistentes (aprendiendo cómo funcionan los volúmenes en Docker, cómo vincularlos correctamente a los contenedores de PostgreSQL, cómo mantener la persistencia de la información incluso después de reiniciar o eliminar contenedores, y comprendiendo las buenas prácticas para la administración y respaldo de datos en entornos virtualizados).

Dificultades de otro tipo:



1. **Curva de aprendizaje:** Al ser un proyecto intermodular, la necesidad de dominar simultáneamente dos frameworks de alto nivel (Angular y Spring) supuso un reto de gestión del tiempo y autoaprendizaje.
2. **Planificación:** Ajustar los tiempos de desarrollo de la lógica del backend con el diseño de la interfaz en Figma para que el progreso fuera paralelo y no se generaran cuellos de botella en la fase de integración.

8. PROPUESTAS DE MEJORA Y POSIBLES AMPLIACIONES

A pesar de que el sistema cumple con todos los requisitos funcionales establecidos, la naturaleza escalable de la arquitectura permite plantear diversas mejoras para futuras versiones:

- **Sincronización Multiusuario en Tiempo Real:** Implementación de **WebSockets** para permitir que las listas de la compra sean colaborativas. Esto permitiría que varios miembros de una familia o unidad conviviente editen la misma lista simultáneamente, visualizando los cambios (añadir o tachar productos) al instante sin necesidad de refrescar la página.
- **Sistema de Alertas de Precios:** Aprovechando el motor de **Scraping**, se podría programar una tarea en segundo plano (*Job Scheduling*) que verifique diariamente los precios de los productos guardados. El sistema notificaría al usuario mediante correo electrónico o notificaciones push cuando un artículo de su interés alcance un precio mínimo histórico.
- **Aplicación Móvil Nativa:** El diseño actual es *responsive*, pero una ampliación lógica sería el desarrollo de una app nativa en **Flutter o Ionic**. Al haber construido una API REST desacoplada, el backend actual serviría sin modificaciones para alimentar la aplicación móvil, permitiendo funciones como el escaneo de códigos de barras mediante la cámara.
- **Integración de Inteligencia Artificial (IA):** Implementar un motor de recomendaciones que, analizando los hábitos de compra y las listas anteriores, sugiera al usuario productos relacionados o le avise si suele olvidar un artículo recurrente en sus compras mensuales.

9. BIBLIOGRAFÍA Y RECURSOS ON-LINE



La elaboración de este proyecto ha requerido la consulta de diversas fuentes técnicas y documentación oficial:

1. **Oracle Java Documentation:** Referencia oficial para el uso de la JDK 17 y flujos de datos. <https://docs.oracle.com/en/java/>
2. **Spring Boot Framework Reference:** Guía para la configuración de Beans, Security y JPA. <https://docs.spring.io/spring-boot/docs/current/reference/html/>
3. **Angular.io Docs:** Documentación sobre la gestión de componentes, servicios e inyección de dependencias. <https://angular.io/docs>
4. **Baeldung Spring Security:** Tutoriales avanzados sobre la implementación de filtros JWT. <https://www.baeldung.com/security-spring>
5. **Docker Documentation:** Guía para la creación de archivos Dockerfile y orquestación con Compose. <https://docs.docker.com/>
6. **Stack Overflow:** Consulta de resolución de errores técnicos específicos durante la fase de integración.

10. GLOSARIO DE TÉRMINOS

10.1. Informáticos

- **Angular:** Framework de desarrollo para aplicaciones web SPA basado en TypeScript.
- **BCrypt:** Algoritmo de hashing salteado diseñado para proteger contraseñas.
- **Docker:** Plataforma de software que permite crear, probar y desplegar aplicaciones rápidamente mediante contenedores.
- **JWT (JSON Web Token):** Método para representar de forma segura pretensiones entre dos partes.
- **PostgreSQL:** Sistema de gestión de bases de datos relacional de código abierto.
- **Spring Boot:** Framework de Java diseñado para simplificar el proceso de creación de aplicaciones basadas en Spring de grado producción.
- **API REST Desacoplada:** Arquitectura de software donde el Backend y el Frontend son independientes, permitiendo que la misma lógica de servidor sirva para una web, una App móvil o servicios de terceros.
- **Flutter / Ionic:** Frameworks de desarrollo multiplataforma que permiten crear aplicaciones nativas para Android e iOS utilizando un único código base.
- **Job Scheduling (Tareas programadas):** Proceso que permite ejecutar funciones automáticamente en momentos específicos (como revisar precios cada madrugada) sin intervención humana.
- **Notificaciones Push:** Mensajes que se envían directamente al dispositivo del usuario (móvil o navegador) para alertar de eventos importantes en tiempo real.
- **Responsive Design:** Técnica de diseño web que busca la correcta visualización de una misma página en distintos dispositivos (desde ordenadores a móviles).
- **WebSockets:** Protocolo de comunicación que permite un flujo de datos bidireccional y constante entre cliente y servidor, ideal para funciones colaborativas en tiempo real.
-



10.2. Problema

- **DataSeeder:** Herramienta interna creada para cargar el inventario inicial y usuarios de prueba automáticamente.
- **LinkedList (Nombre comercial):** Marca del proyecto que simboliza la unión y organización de productos de compra.
- **Scraper:** Módulo especializado en la lectura y extracción de etiquetas HTML para obtener metadatos de productos.
- **Sincronización Multiusuario:** Capacidad del sistema para gestionar ediciones simultáneas en una misma lista por parte de varios miembros de un grupo familiar.
- **Motor de Recomendaciones (IA):** Algoritmo basado en Inteligencia Artificial que analiza el historial del usuario para predecir necesidades de compra futuras.

11. ANEXOS

- **Anexo I: Guía de Estilos.** Incluye la paleta de colores corporativa, tipografías y el diseño de logotipos realizado en [Figma](#).
- **Anexo II: Manual de Usuario.** Guía visual con capturas de pantalla de la aplicación real funcionando: flujo de registro, creación de listas y visualización de productos. [Aquí](#)
- **Anexo III: Configuración de Entorno.** Listado completo del archivo [docker-compose.yml](#) y variables de entorno necesarias para el despliegue.
- **Anexo IV: [Presentación](#).** Donde se muestran las funcionalidades principales y razonamiento y problema que busca solucionar nuestro proyecto.
- **Anexo V:** Ejecutable para ejilizar el trabajo de la [terminal](#)

